

Universidad Mariano Gálvez De Guatemala

Centro Universitario Santa Cruz Del Quiché

Ingeniería en Sistemas de Información y Ciencias de Computación.

Ing. Marcos Urizar

VI Ciclo.

Proyecto de Sistema Web de Ventas e Inventario

Estudiantes y sus roles:

Sebastián Eduardo de León Gómez: Administración Linux, Azure, Nginx, Tomcat y despliegue

1090-24-8418

Adolfo Arturo Flores Franco: Desarrollo Java, PostgreSQL y lógica del sistema

1090-24-5361

Alejandro José Amadeo Garzona Álvarez: Diseño de interfaz e integración visual
1090-24-5466

Cristhel Ariana Reynoso Catalán: documentación y evidencias

1090-24-10422

4 de noviembre de 2026.

Índice

1. Introducción 3
2. Objetivos 3
3. Alcance y metodología 3
4. Descripción general y arquitectura 4
5. Infraestructura Linux y despliegue 4
6. Backend Java y base de datos PostgreSQL 6
7. Verificación operativa, monitoreo y seguridad 8
8. Matriz de pruebas funcionales 9
9. Respaldo y restauración 10
10. Gestión de evidencias 11
11. Resultados y estado del proyecto 12
12. Problemas, riesgos y recomendaciones 12
13. Anexos gráficos y fuentes 12
14. Pruebas funcionales: usuario Administrador 36
15. Pruebas funcionales: usuario Operador 42
16. Evidencias del frontend y scripts PostgreSQL 44
17. Conclusiones 48

1. Introducción

La presente documentación técnica final presenta el proyecto integrador «Sistema Web de Ventas e Inventario», desarrollado para el curso de Sistemas Operativos. La iniciativa integra una aplicación web Java con una base de datos relacional PostgreSQL y una infraestructura de publicación sobre Ubuntu Server en Microsoft Azure. Además de implementar funciones de autenticación y administración de productos, el proyecto permite aplicar conocimientos de administración de servicios, acceso remoto, despliegue de aplicaciones, control de permisos, supervisión de recursos y recuperación de información.

La documentación reúne los aportes técnicos del equipo en sus cuatro áreas de trabajo: infraestructura Linux y publicación en el servidor; backend y base de datos PostgreSQL; frontend; y pruebas, monitoreo y documentación. A partir de estos materiales se organiza una descripción verificable de la arquitectura, los procedimientos de configuración, el modelo de datos y las evidencias de funcionamiento.

Las capturas se identifican según su procedencia: entorno local de desarrollo o servidor publicado en Microsoft Azure.

2. Objetivos

2.1 Objetivo general

Documentar y evaluar técnicamente el sistema web de ventas e inventario mediante el análisis de su infraestructura Ubuntu, su backend Java, su base de datos PostgreSQL y su frontend, así como de las pruebas funcionales y los procedimientos de monitoreo y recuperación de datos.

2.2 Objetivos específicos

• Describir la configuración del VPS y los servicios necesarios para publicar la aplicación.

• Explicar la estructura del backend, el modelo relacional y las operaciones sobre productos.

• Organizar las evidencias técnicas del proyecto y diferenciar las pruebas locales de las realizadas en el servidor.

• Ejecutar y registrar una matriz de pruebas funcionales y describir los procedimientos de monitoreo, respaldo y restauración.

• Identificar riesgos técnicos y formular recomendaciones para la operación del sistema.

3. Alcance y metodología

Se revisaron las evidencias de configuración del servidor Linux (17 capturas), el informe técnico del backend y PostgreSQL (29 capturas, 39 páginas), las capturas del frontend y las pruebas funcionales realizadas desde el navegador con los perfiles Administrador y

Operador. El informe incorpora las descripciones, comandos y resultados expresamente documentados en esos materiales.

Fuente	Cobertura	Observaciones
Evidencias de infraestructura Linux	Azure, SSH, Java, Tomcat, Nginx, GitHub, despliegue y monitoreo	Estado de red descrito en sección 5.6
Informe técnico de backend y PostgreSQL	PostgreSQL, esquema, scripts, JDBC, DAO, seguridad y 29 capturas	Procedimiento de restauración documentado
Capturas de frontend y pruebas funcionales	Pantallas, perfiles Administrador y Operador y operaciones sobre productos	Evidencia de operaciones en interfaz

4. Descripción general y arquitectura

El usuario accede a la aplicación desde un navegador. Nginx recibe solicitudes HTTP y las redirige al servidor Apache Tomcat, que ejecuta el archivo WAR del sistema. La aplicación Java organiza la lógica mediante páginas JSP, controladores Servlet, objetos de acceso a datos (DAO) y conectividad JDBC hacia PostgreSQL. Esta separación facilita el mantenimiento de la interfaz, las reglas de negocio y la persistencia.

Capa	Tecnología / función
Cliente	Navegador web; formularios HTML/JSP
Proxy inverso	Nginx en puerto 80; reenvío a 127.0.0.1:8080
Servidor de aplicaciones	Apache Tomcat 10; aplicación SistemaWebVentasInventario.war
Lógica de negocio	Java 17; Servlets, modelos y validaciones
Persistencia	DAO y JDBC con controlador PostgreSQL
Datos	PostgreSQL; tablas rol, usuario y productos
Alojamiento	Máquina virtual Ubuntu 24.04.5 LTS en Microsoft Azure

Flujo lógico: Navegador → Nginx → Tomcat/Servlets → DAO/JDBC → PostgreSQL. Las respuestas regresan por la misma ruta hacia el navegador.

5. Infraestructura Linux y despliegue

5.1 Entorno de alojamiento

Se configuró una máquina virtual denominada vm-sistema-ventas, alojada en Microsoft Azure, con Ubuntu Server 24.04.5 LTS, 2 vCPU y 4 GB de memoria RAM. La IP pública registrada es 57.156.71.74. La administración se realiza mediante el usuario sysadmin y conexiones SSH autenticadas por clave privada. La máquina virtual pertenece al grupo de recursos rg-sistema-ventas.

5.2 Acceso remoto y privilegios

El acceso remoto se verificó desde Git Bash en Windows. Los comandos whoami, hostname, id y sudo -l permitieron identificar la sesión, el servidor y los privilegios administrativos. La documentación de despliegue describe una clave SSH privada, cuyo contenido no debe publicarse ni reproducirse en el informe. La conexión se establece con el comando «ssh -i ~/.ssh/<clave_privada> sysadmin@57.156.71.74».

5.3 Herramientas y servicios

La preparación del entorno comprende OpenJDK 17, Maven 3.8.7, Apache Tomcat 10, Nginx, PostgreSQL 16 y Git. Nginx funciona como proxy inverso y Tomcat como contenedor de la aplicación Java. El código se obtuvo desde el repositorio privado wallo56/Sistema-Web-Ventas-Inventario, inicialmente a partir de la rama feature/backend. El proyecto se alojó en /home/sysadmin/proyectos/Sistema-Web-Ventas-Inventario/ y el acceso del servidor al repositorio privado se configuró con una clave de despliegue de solo lectura, definida en el archivo ~/.ssh/config, sin incorporar credenciales al código fuente.

5.4 Construcción y publicación

El proyecto se compiló mediante «mvn clean package», generando SistemaWebVentasInventario.war. Posteriormente, el archivo se desplegó en /var/lib/tomcat10/webapps/ y se comprobó el servicio tomcat10 con systemctl. El sitio Nginx «sistema-ventas» recibe tráfico en el puerto 80 y lo reenvía a Tomcat en 127.0.0.1:8080. La compilación se ejecutó desde el directorio SistemaWebVentasInventario/ del repositorio y, después de copiar el WAR, se reinició tomcat10 para cargar la aplicación. La configuración de Nginx se encuentra en /etc/nginx/sites-available/sistema-ventas, está habilitada mediante un enlace simbólico en /etc/nginx/sites-enabled/ y conserva las cabeceras del host, la dirección IP del cliente y el protocolo de la solicitud.

5.5 Configuración de credenciales

La conexión a PostgreSQL se configura mediante variables de entorno, entre ellas DB_URL y DB_PASSWORD, conservadas fuera del repositorio en /etc/sistema-ventas/tomcat.env. Esta práctica reduce la exposición accidental de secretos en el código fuente y la documentación. El servicio Tomcat se configuró mediante systemd para cargar estas variables al iniciarse. En el VPS se utiliza PostgreSQL 16 con la base sistema_inventario_web y el usuario de aplicación app_inventario.

5.6 Estado de red y publicación

La revisión del firewall con el comando «sudo ufw status verbose» devolvió el resultado «inactive». Esto no demuestra ausencia total de filtrado de red, porque Azure dispone de reglas independientes, pero sí exige revisar la protección efectiva del VPS. El sistema se publica actualmente mediante HTTP; la configuración de HTTPS queda pendiente.

Según el procedimiento de despliegue, después de publicar la aplicación se comprobaron el acceso desde un navegador externo, el inicio de sesión con un usuario

válido, el rechazo de credenciales incorrectas, la protección de páginas mediante sesiones, el registro, la consulta, la modificación y la desactivación lógica de productos, la validación de precios y existencias negativos, la persistencia de los registros después de reiniciar Tomcat y el arranque automático del servicio. Estas comprobaciones se reportaron como satisfactorias; las que no cuentan con captura se identifican en la matriz de la sección 8.

Dirección del sistema: http://57.156.71.74/SistemaWebVentasInventario/login

6. Backend Java y base de datos PostgreSQL

6.1 Organización del backend

El backend sigue una arquitectura Java Web basada en Servlets, páginas JSP, clases de modelo y DAO. ConexionBD establece la comunicación JDBC; UsuarioDAO y ProductoDAO ejecutan consultas; SeguridadPassword verifica hashes de contraseñas. Se implementan controles de sesión, autorización por rol y protección CSRF.

6.2 Base de datos y ambientes

La base de datos se denomina sistema_inventario_web y utiliza el esquema public. En el entorno local de desarrollo se utiliza PostgreSQL 17 y en el VPS, PostgreSQL 16. Esta diferencia de versión debe considerarse al validar la compatibilidad y los procedimientos de recuperación. La administración local se realizó con pgAdmin 4 sobre localhost:5432, y la aplicación se conecta con el rol de base de datos app_inventario.

6.3 Modelo entidad-relación



Modelo entidad-relación de la base de datos sistema_inventario_web. Fuente: informe técnico del backend.

La relación entre rol y usuario es de uno a muchos: un rol puede asignarse a varios usuarios. La tabla productos es independiente en el modelo documentado. Los estados

A e I permiten representar registros activos e inactivos; precio y stock tienen restricciones para impedir valores negativos.

6.4 Scripts y carga de datos

Los scripts se publicaron en la carpeta database/ del repositorio (rama feature/backend, commit 94f3d08): database/schema.sql, database/datos_prueba.sql y database/permisos.sql. El primero crea las tablas y sus restricciones; el segundo incorpora los roles ADMINISTRADOR y OPERADOR y cinco productos de ejemplo para un despliegue nuevo; el tercero concede permisos limitados al usuario de aplicación app_inventario. Las cuentas de usuario de prueba se preparan en un archivo privado fuera del repositorio, para no publicar sus hashes. Las capturas del entorno local muestran diez productos y cambios posteriores, por corresponder a momentos y ambientes distintos. El contenido de los scripts se presenta en la sección 16.2.

6.5 Funcionalidad y seguridad

Las evidencias del backend incluyen conexión JDBC exitosa, autenticación del usuario admin con rol ADMINISTRADOR, formulario de inicio de sesión, dashboard, listado, creación, actualización y desactivación lógica de productos. Al intentar eliminar un producto sin permisos, Tomcat responde HTTP 403 – Forbidden con el mensaje «No tienes permisos para eliminar productos», lo que demuestra que la autorización se aplica en el servidor. Las contraseñas se almacenan con PBKDF2WithHmacSHA256, sal aleatoria de 16 bytes, 600 000 iteraciones y clave de 256 bits, con el formato PBKDF2-SHA256$iteraciones$sal$hash. Las credenciales reales no se incluyen en este informe. Las capturas correspondientes se presentan en la sección 13.2.

6.6 Componentes verificados en el WAR

Se comprobó la presencia de los siguientes componentes dentro del archivo SistemaWebVentasInventario.war:

Componente	Qué representa
WEB-INF/web.xml	Configuración de la aplicación web
postgresql-42.7.7.jar	Driver JDBC de PostgreSQL dentro de WEB-INF/lib
ConexionBD.class	Capa de conexión JDBC
UsuarioDAO.class / ProductoDAO.class	Acceso a datos y operaciones CRUD
Usuario.class / Producto.class	Entidades del modelo
SeguridadPassword.class	Verificación de hashes y manejo de contraseñas
Servlets (p. ej. ActualizarProductoServlet.class)	Controladores web

6.7 Rutas principales

Las rutas se documentan en el contexto /SistemaWebVentasInventario. La vista está compuesta por index.html, login.jsp y páginas JSP ubicadas bajo WEB-INF.

Ruta relativa	Uso / evidencia
/login	Formulario de inicio de sesión
/dashboard	Panel autenticado
/productos	Listado y acciones de gestión
/productos/crear	Registro de productos
/productos/actualizar	Actualización de productos
/productos/eliminar	Acción protegida; responde HTTP 403 sin permisos
/logout	Cierre de sesión

6.8 Consultas de verificación

Las siguientes consultas de solo lectura se ejecutan en pgAdmin 4 (Query Tool) para comprobar la base, los roles, los usuarios sin mostrar hashes y el inventario:

```
SELECT current_database() AS base_datos, current_user AS usuario_bd;
SELECT table_name FROM information_schema.tables
  WHERE table_schema = 'public' AND table_type = 'BASE TABLE';
SELECT id_rol, nombre FROM public.rol ORDER BY id_rol;
SELECT u.id_usuario, u.usuario, u.estado, r.nombre AS rol
  FROM public.usuario u JOIN public.rol r ON u.id_rol = r.id_rol;
SELECT id_producto, nombre, precio, stock, estado
  FROM public.productos ORDER BY id_producto;
SELECT COUNT(*) AS productos_activos, COALESCE(SUM(stock),0) AS unidades
  FROM public.productos WHERE estado = 'A';
```

En las capturas se observa el listado de diez productos locales, el producto temporal de prueba, su paso a estado inactivo y los productos agregados y actualizados desde la interfaz, confirmados en PostgreSQL.

7. Verificación operativa, monitoreo y seguridad

La supervisión del servidor la realiza el administrador del VPS con el usuario sysadmin, mediante una conexión SSH con autenticación por clave privada. El procedimiento consiste en conectarse al servidor, ejecutar los comandos de consulta, comprobar el estado de los servicios y el consumo de recursos, guardar los resultados y revisarlos antes de compartirlos, para que no contengan contraseñas, tokens ni claves privadas. Los comandos son de solo consulta y no modifican la configuración del servidor.

Comando	Finalidad	Evidencia requerida
free -h	Memoria RAM utilizada y disponible	Salida de terminal
df -h	Espacio usado y disponible en disco	Salida de terminal
uptime	Tiempo de actividad y carga	Salida de terminal

systemctl status nginx --no-pager	Estado del proxy inverso	Servicio activo / errores
systemctl status tomcat10 --no-pager	Estado del servidor de aplicaciones	Servicio activo / errores
systemctl status postgresql@16-main --no-pager	Estado del motor de datos en VPS	Servicio activo / errores
systemctl is-active nginx tomcat10 postgresql@16-main	Comprobar qué servicios están activos	Salida de terminal
journalctl -u tomcat10 -n 20 --no-pager	Eventos recientes de Tomcat	Extracto depurado sin secretos

7.1 Script de monitoreo

El archivo deploy/verificar_servidor.sh está previsto para consultar la memoria, el disco y el estado de los servicios Nginx, Tomcat y PostgreSQL. Su sintaxis puede verificarse con «bash -n deploy/verificar_servidor.sh» y su ejecución periódica permite detectar fallas de servicio de forma temprana.

8. Matriz de pruebas funcionales

La matriz resume los resultados de las pruebas realizadas desde el navegador sobre la aplicación integrada, con los perfiles Administrador y Operador. Las evidencias se detallan en las secciones 14 y 15. El estado «Reportada» indica una prueba informada como satisfactoria en el procedimiento de despliegue del VPS, para la cual no se dispone de captura.

Prueba / acción	Resultado esperado	Resultado obtenido	Estado
Inicio de sesión válido — Ingresar usuario y contraseña correctos	Acceso al dashboard	Acceso al dashboard con las cuentas admin y operador	Aprobada
Inicio de sesión inválido — Ingresar contraseña incorrecta	Rechazo y mensaje de error	Reportada como satisfactoria en el despliegue del VPS; sin captura	Reportada
Acceso sin sesión — Abrir ruta protegida sin autenticarse	Redirección o bloqueo	Protección de páginas por sesión reportada como satisfactoria en el VPS; sin captura	Reportada
Listado de productos — Abrir módulo de productos	Mostrar registros almacenados	Se muestran los productos con código, precio y existencias	Aprobada
Crear producto — Enviar formulario con datos válidos	Registro visible y persistente	Se registraron P-010 (Memoria RAM) y P-011 (Cámara Web HD)	Aprobada
Editar producto — Modificar precio o existencias	Cambios guardados	P-004 pasó de Q100 y 25 unidades a Q150 y 50 unidades	Aprobada

Desactivar producto — Confirmar eliminación lógica	Estado inactivo	P-010 dejó de mostrarse en el listado	Aprobada
Precio negativo — Ingresar precio menor que cero	Rechazo de valor	Restricción CHECK en el esquema; validación reportada como satisfactoria en el VPS, sin captura	Reportada
Stock negativo — Ingresar existencias menores que cero	Rechazo de valor	Restricción CHECK en el esquema; validación reportada como satisfactoria en el VPS, sin captura	Reportada
Acción sin permisos — Intentar operación restringida	Respuesta de acceso denegado	HTTP 403 en el backend; el Operador no ve la opción Eliminar	Aprobada
Cerrar sesión — Utilizar opción de salir	Sesión finalizada	Retorno al formulario de inicio de sesión	Aprobada
Persistencia tras reinicio — Consultar registros tras reiniciar Tomcat	Datos conservados	Cambios confirmados en PostgreSQL local; persistencia tras reiniciar Tomcat reportada en el VPS, sin captura	Reportada

9. Respaldo y restauración

9.1 Estrategia de respaldo

El respaldo de PostgreSQL se realiza mediante pg_dump. Para el formato personalizado se utiliza «-F c» y se restaura con pg_restore; para un respaldo SQL plano se utiliza psql. El archivo debe guardarse en un lugar privado, pues puede contener información sensible, incluidos hashes de contraseñas.

```
# En el VPS (Ubuntu)
sudo -u postgres pg_dump -F c -b -v \
  -f /var/backups/sistema_inventario_web.backup sistema_inventario_web
# En el entorno local (Windows, PostgreSQL 17)
& "C:\Program Files\PostgreSQL\17\bin\pg_dump.exe" -h localhost -p 5432 `
  -U postgres -d sistema_inventario_web -F c -b -v `
  -f "$HOME\Documents\sistema_inventario_web.backup"
```

9.2 Restauración en entorno aislado

La restauración debe efectuarse en una base de prueba separada, por ejemplo sistema_inventario_prueba, nunca sobre la base de producción. Primero se crea la base vacía y luego se restaura el respaldo:

```
createdb -h localhost -p 5432 -U postgres sistema_inventario_prueba
pg_restore -h localhost -p 5432 -U postgres \
  -d sistema_inventario_prueba --no-owner --no-acl -v sistema_inventario_web.backup
# Si el respaldo es de texto plano (.sql):
psql -h localhost -p 5432 -U postgres -d sistema_inventario_prueba -f
sistema_inventario_web.sql
```

Las opciones --no-owner y --no-acl evitan depender de los propietarios y permisos del ambiente de origen; después se deben volver a conceder los permisos de permisos.sql. Para comprobar una restauración se consultan las tablas, los roles, los usuarios y los productos mediante las consultas de la sección 6.8. Este apartado describe el procedimiento, no una restauración ejecutada.

9.3 Registro de resultados

Control	Dato por registrar
Base de origen	sistema_inventario_web
Fecha y hora del respaldo	Campo para completar durante la ejecución
Formato y ruta del archivo	Campo para completar durante la ejecución
Tamaño del respaldo	Pendiente
Base de restauración	Campo para completar durante la ejecución
Conteos antes / después	Campo para completar durante la ejecución
Capturas de ejecución	Campo para completar durante la ejecución

10. Gestión de evidencias

La entrega exige al menos 20 capturas pertinentes, con título, descripción y correspondencia con la actividad técnica. El proyecto reúne 17 evidencias de infraestructura Linux, 29 del backend y capturas del frontend y de las pruebas funcionales. La siguiente tabla resume las evidencias documentadas por área de trabajo.

Área	Información disponible	Evidencias documentadas
Infraestructura Linux	Acceso SSH, servidor, servicios, despliegue y procedimiento de monitoreo	Acceso SSH, servicios y despliegue
Backend y PostgreSQL	Esquema, scripts SQL, arquitectura, evidencias locales y procedimiento de respaldo	Esquema, consultas y procedimientos SQL
Frontend	Pantallas, perfiles Administrador y Operador y flujo de navegación	Pantallas y flujo de navegación
Pruebas y documentación	Matriz de pruebas ejecutada, pruebas por perfil e informe consolidado	Matriz y pruebas por perfil

11. Resultados y estado del proyecto

El backend está desplegado en el VPS de Azure detrás de Nginx y Tomcat, y se documentaron verificaciones de acceso, servicios, autenticación y gestión de productos. El frontend está integrado con el backend y las pruebas desde el navegador muestran el funcionamiento de los perfiles Administrador y Operador. Los procedimientos de respaldo, restauración y supervisión se describen en sus apartados correspondientes.

Componente	Estado sustentado
Servidor Ubuntu y acceso SSH	Configurado y documentado con capturas
Java, Nginx y Tomcat	Configuración y estado documentados
Backend Java y PostgreSQL	Desplegado en el VPS; desarrollo y pruebas locales documentados
Frontend definitivo	Integrado; probado con los perfiles Administrador y Operador
Matriz de pruebas completa	7 pruebas aprobadas con evidencia y 5 verificaciones reportadas sin captura
Respaldo y restauración	Procedimientos de respaldo y restauración descritos
HTTPS y revisión de firewall	Estado de red y recomendaciones de seguridad documentados

12. Problemas, riesgos y recomendaciones

El principal riesgo de seguridad visible es que UFW aparece inactivo y la aplicación se presenta mediante HTTP. Para fortalecer la seguridad del despliegue deben verificarse las reglas de Azure, la exposición de puertos y la configuración de HTTPS. Otro aspecto importante es mantener las credenciales fuera de GitHub, capturas y documentos, tal como ya se plantea con las variables de entorno privadas.

La diferencia entre PostgreSQL 17 local y PostgreSQL 16 en el VPS exige precaución al planificar respaldos y restauraciones; el procedimiento definitivo debe validarse con las versiones y herramientas realmente utilizadas en el servidor.

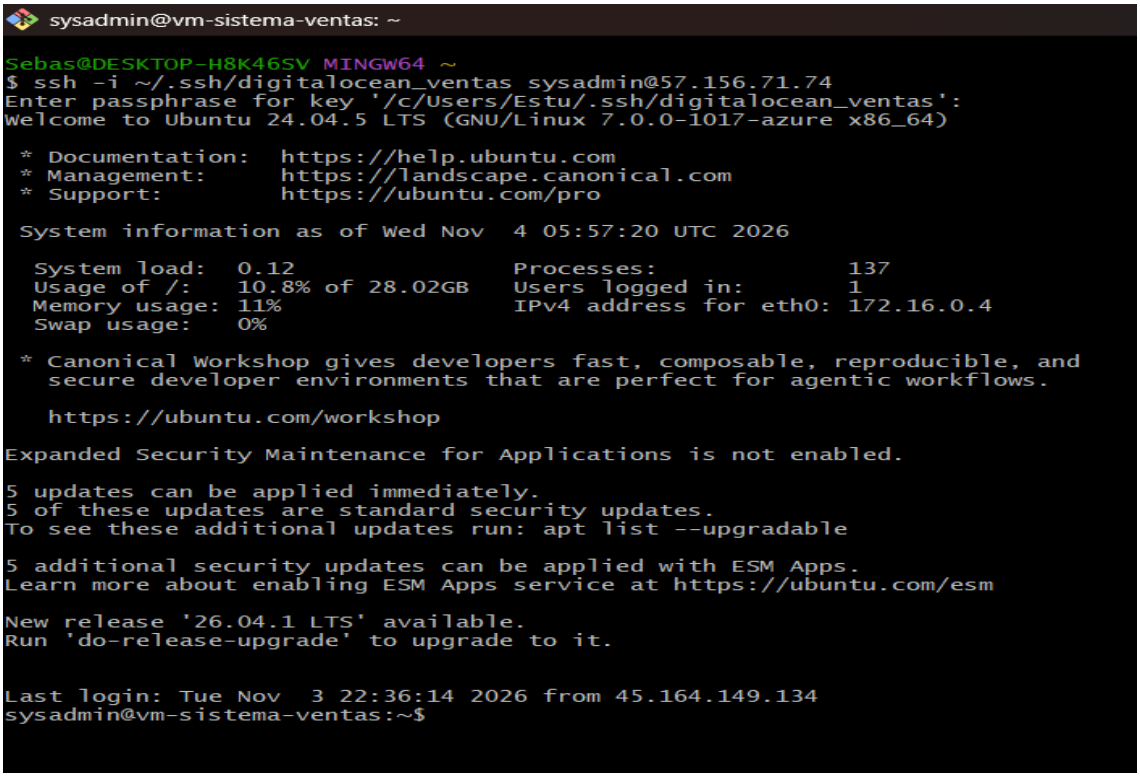
Se recomienda conservar evidencias claras de las verificaciones realizadas y comprobar las restricciones de permisos tanto en la interfaz como en el servidor. También conviene mantener una nomenclatura uniforme para las capturas, asociada a fecha y resultado.

13. Anexos gráficos y fuentes

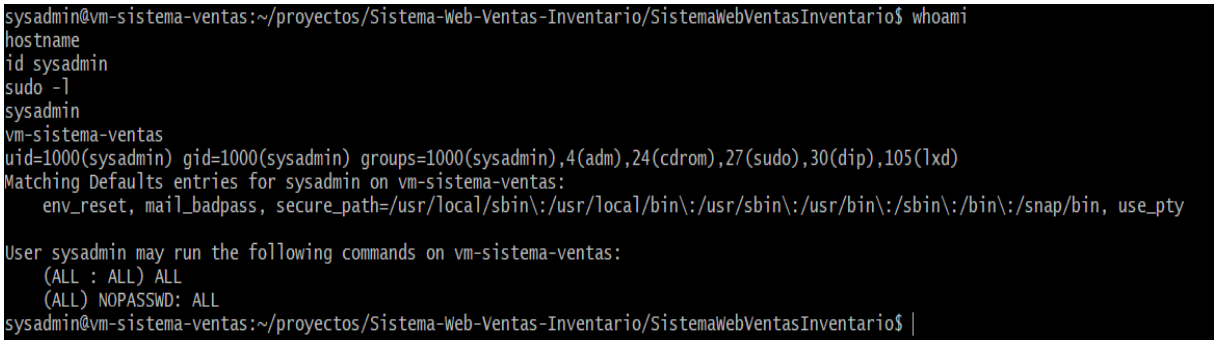
Selección de evidencias originales de la configuración del servidor y del backend, con referencia a su fuente.

13.1 Evidencias de infraestructura Linux

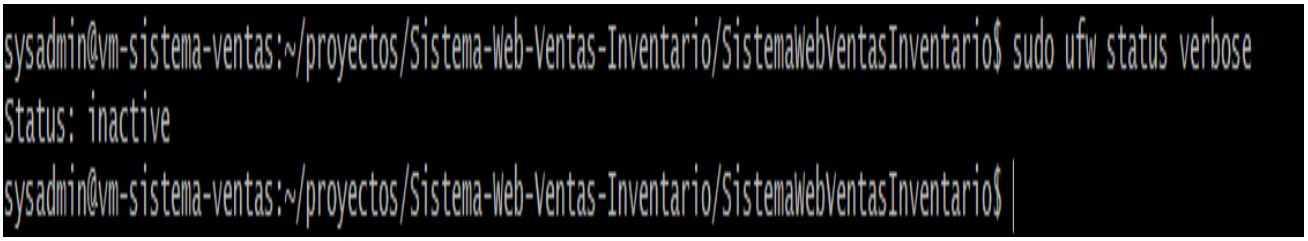
Capturas del acceso, la configuración de servicios y el despliegue en el servidor Ubuntu de Azure, en la secuencia técnica de implementación.



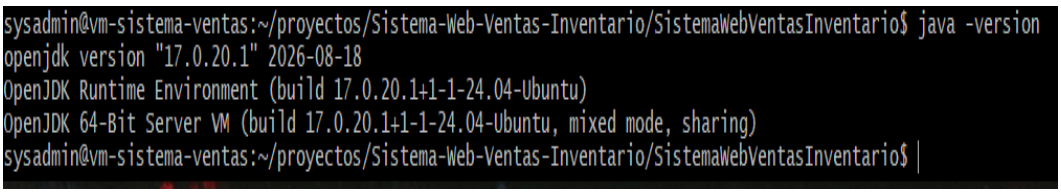
Acceso remoto mediante SSH. Inicio de sesión remoto desde Git Bash de Windows hacia la máquina virtual Ubuntu en Azure; acceso del usuario sysadmin al servidor vm-sistema-ventas. Fuente: evidencias de infraestructura Linux, evidencia 01.



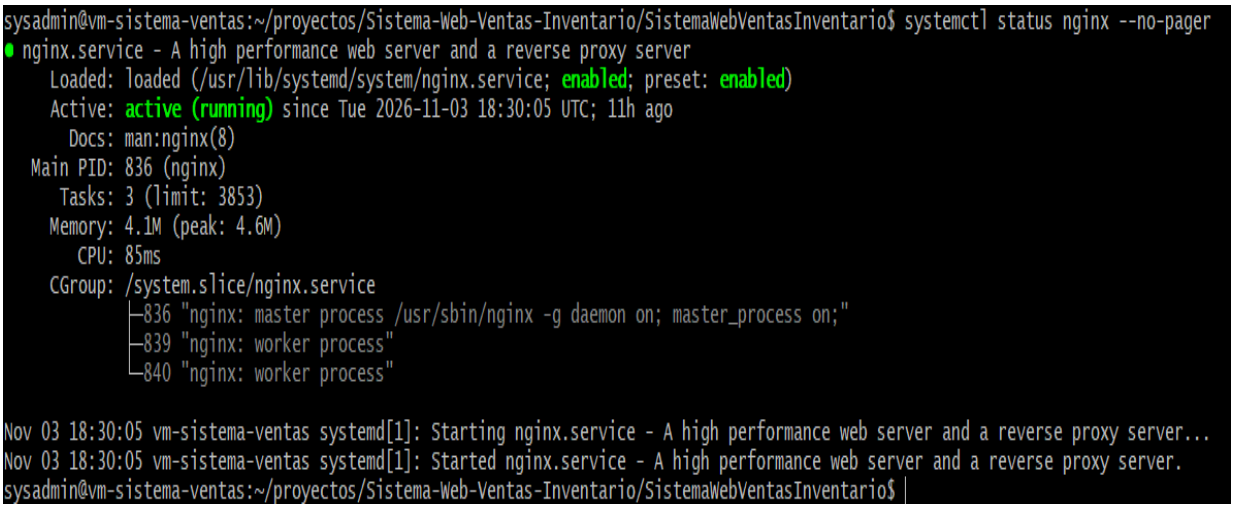
Privilegios del usuario administrador. Los comandos whoami, hostname, id sysadmin y sudo -l muestran que sysadmin pertenece al grupo sudo y cuenta con permisos de administración. Fuente: evidencias de infraestructura Linux, evidencia 02.



Estado del firewall UFW. sudo ufw status verbose muestra Status: inactive; las reglas de red de Azure deben verificarse por separado. Fuente: evidencias de infraestructura Linux, evidencia 03.



Versión de Java. java -version confirma la instalación de OpenJDK 17. Fuente: evidencias de infraestructura Linux, evidencia 04.



Estado del servicio Nginx. systemctl status nginx muestra Nginx en ejecución como proxy inverso. Fuente: evidencias de infraestructura Linux, evidencia 05.



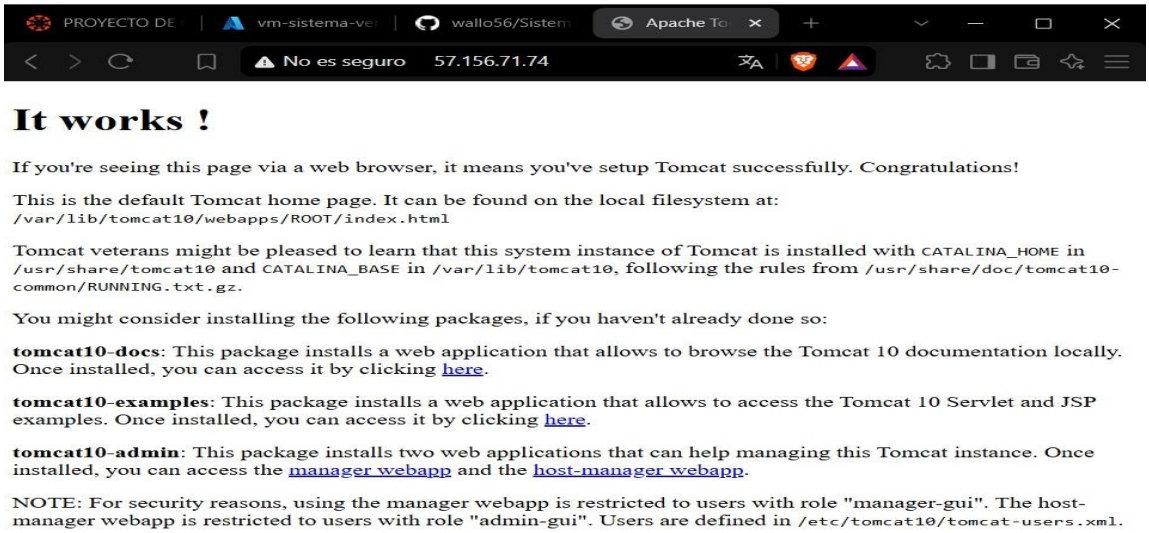
Estado del servicio Tomcat. systemctl status tomcat10 muestra el estado del servidor de aplicaciones. Fuente: evidencias de infraestructura Linux, evidencia 06.

```
sysadmin@vm-sistema-ventas:~/proyectos/Sistema-Web-Ventas-Inventario/SistemaWebVentasInventario$ sudo ls -l /etc/nginx/sites-enabled/
total 0
lrwxrwxrwx 1 root root 41 Nov  3 10:27 sistema-ventas -> /etc/nginx/sites-available/sistema-ventas
sysadmin@vm-sistema-ventas:~/proyectos/Sistema-Web-Ventas-Inventario/SistemaWebVentasInventario$ sudo cat /etc/nginx/sites-available/sistema-ventas
server {
    listen 80;
    server_name 57.156.71.74;

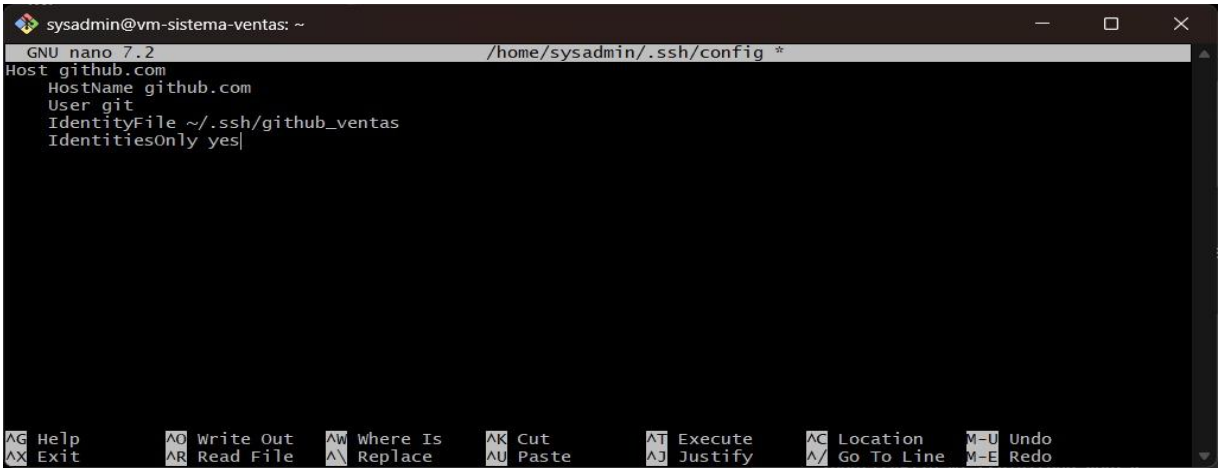
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
sysadmin@vm-sistema-ventas:~/proyectos/Sistema-Web-Ventas-Inventario/SistemaWebVentasInventario$
```

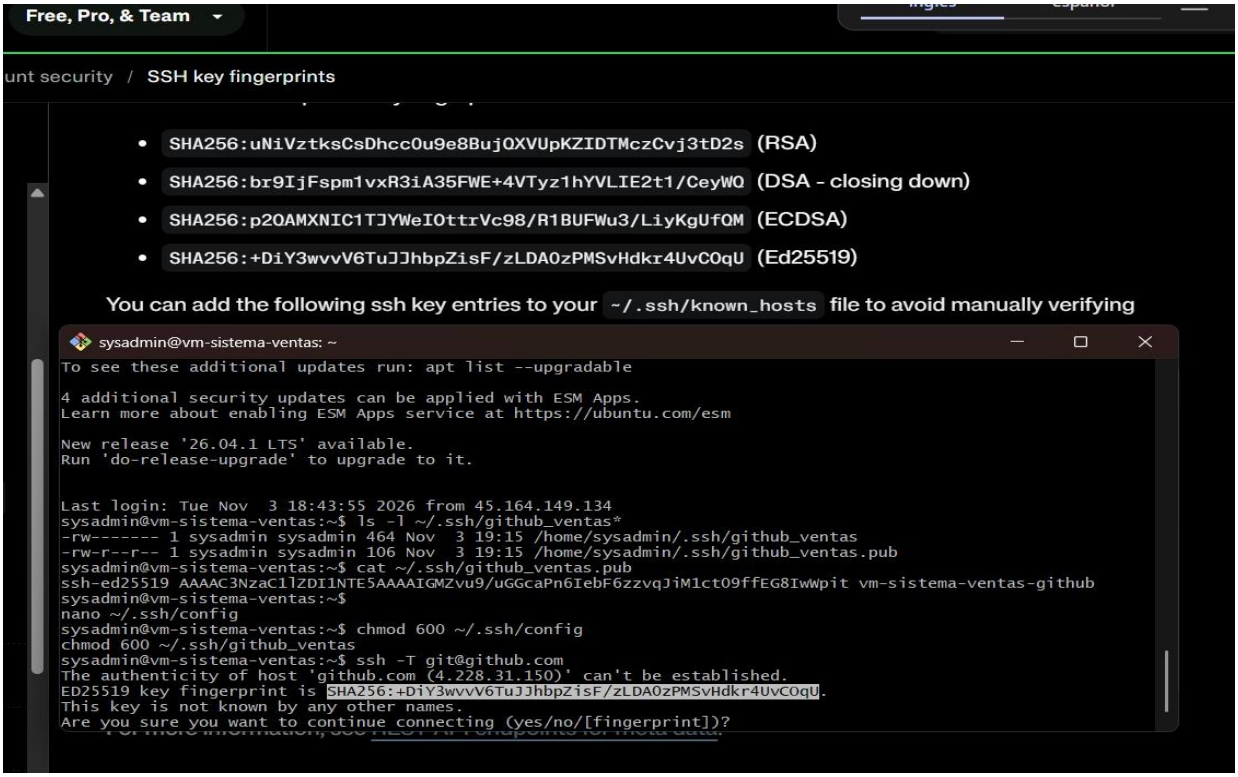
Configuración del proxy inverso Nginx. El sitio sistema-ventas recibe peticiones en el puerto 80 y las reenvía a Tomcat en 127.0.0.1:8080. Fuente: evidencias de infraestructura Linux, evidencia 07.



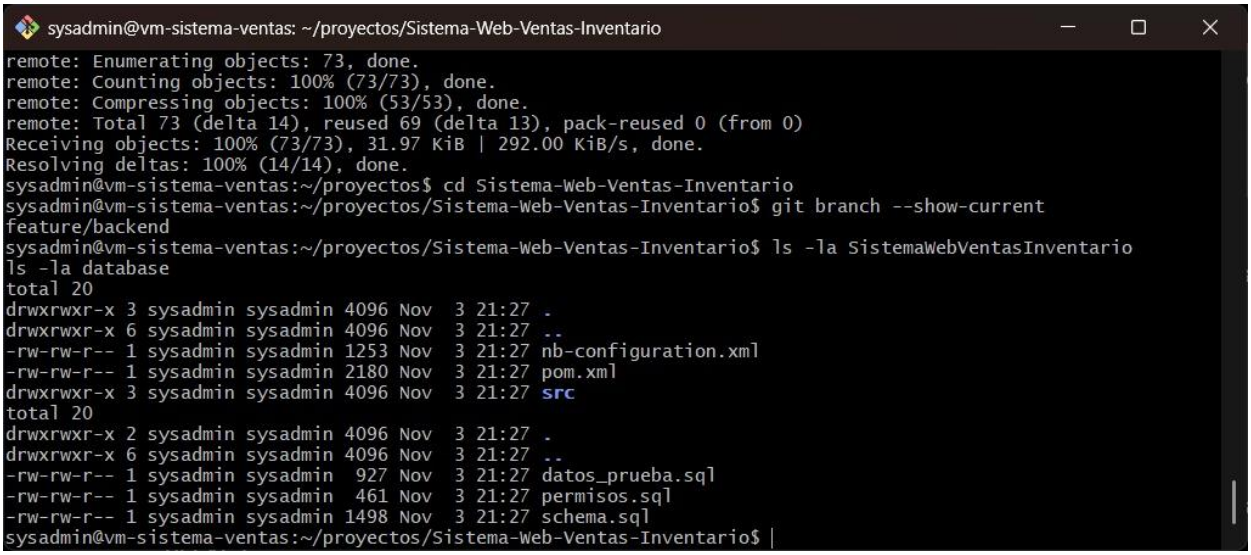
Tomcat accesible desde la IP pública. El navegador muestra la página inicial de Apache Tomcat en 57.156.71.74, lo que confirma que el servidor responde desde Internet. Fuente: evidencias de infraestructura Linux, evidencia 08.



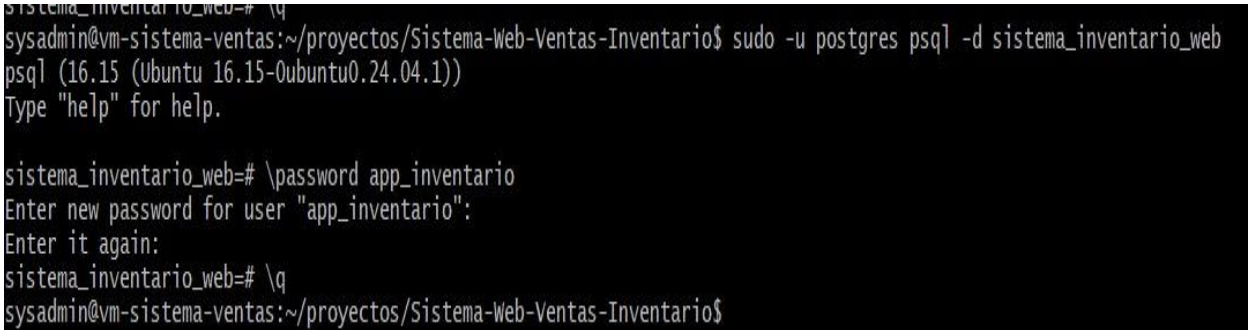
Configuración SSH para Git. Edición de ~/.ssh/config para definir la identidad utilizada al acceder al repositorio privado en GitHub. Fuente: evidencias de infraestructura Linux, evidencia 09.



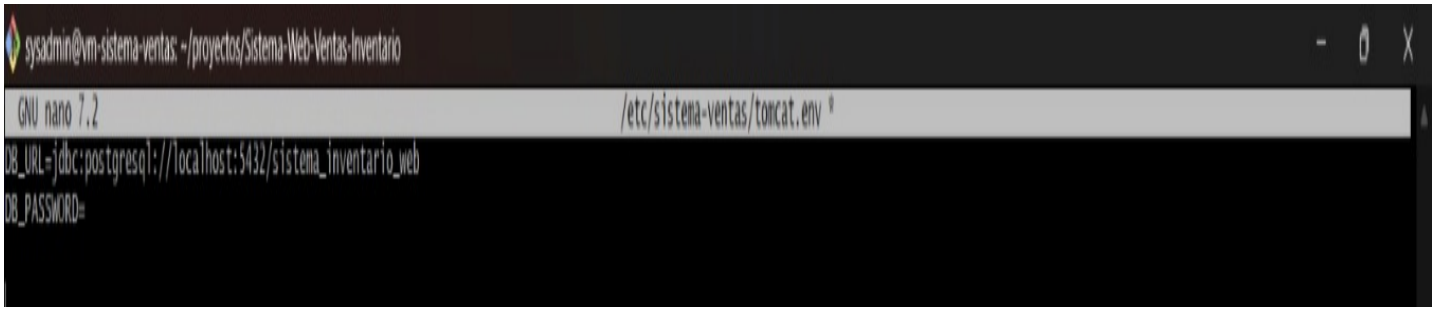
Vinculación del servidor con GitHub. Configuración de la clave SSH de despliegue y verificación de la huella de github.com. Fuente: evidencias de infraestructura Linux, evidencia 10.

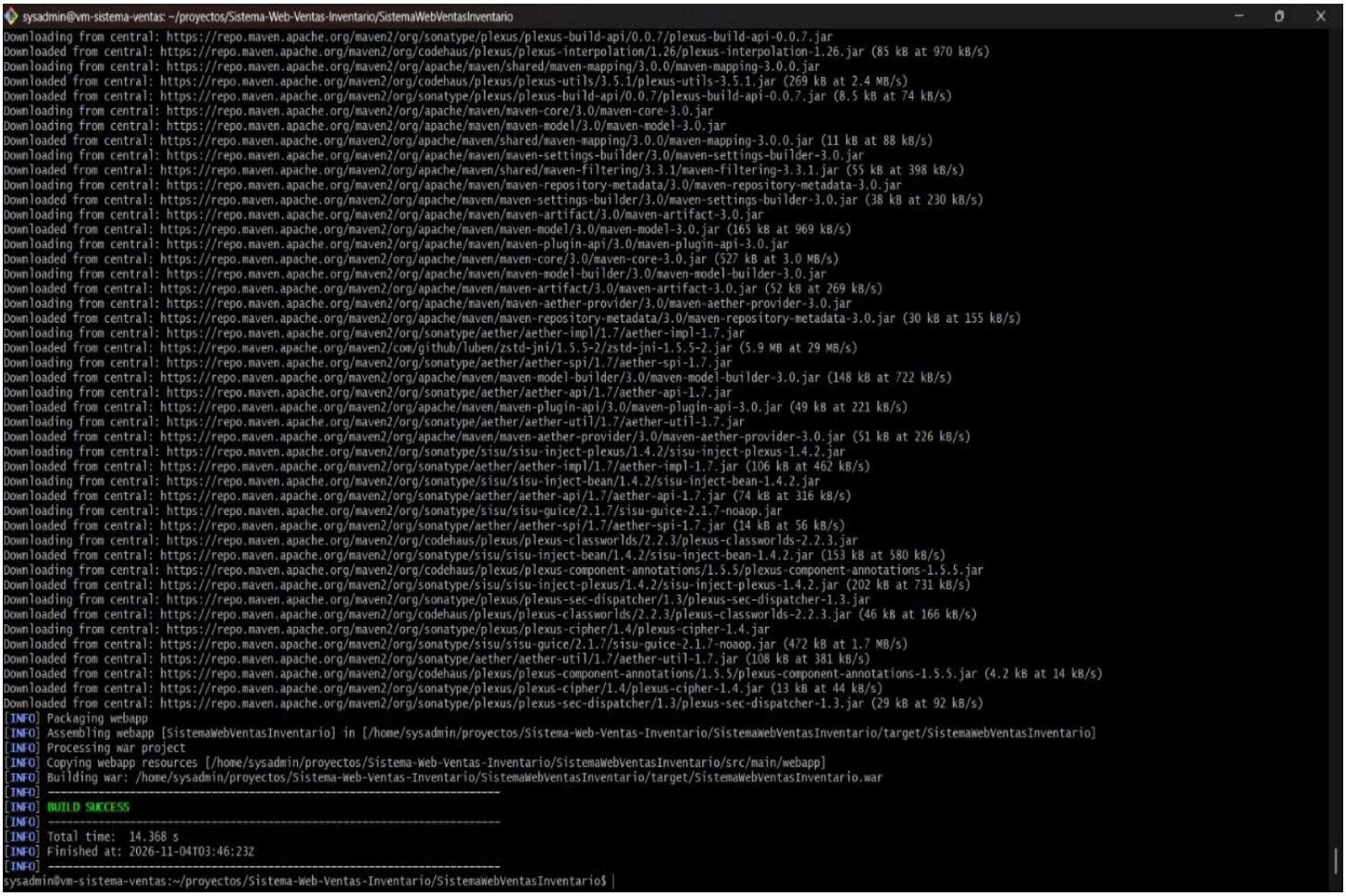


Descarga del backend y archivos del proyecto. Clonación del repositorio en la rama feature/backend y verificación de los archivos, incluida la carpeta database con schema.sql, datos_prueba.sql y permisos.sql. Fuente: evidencias de infraestructura Linux, evidencia 11.

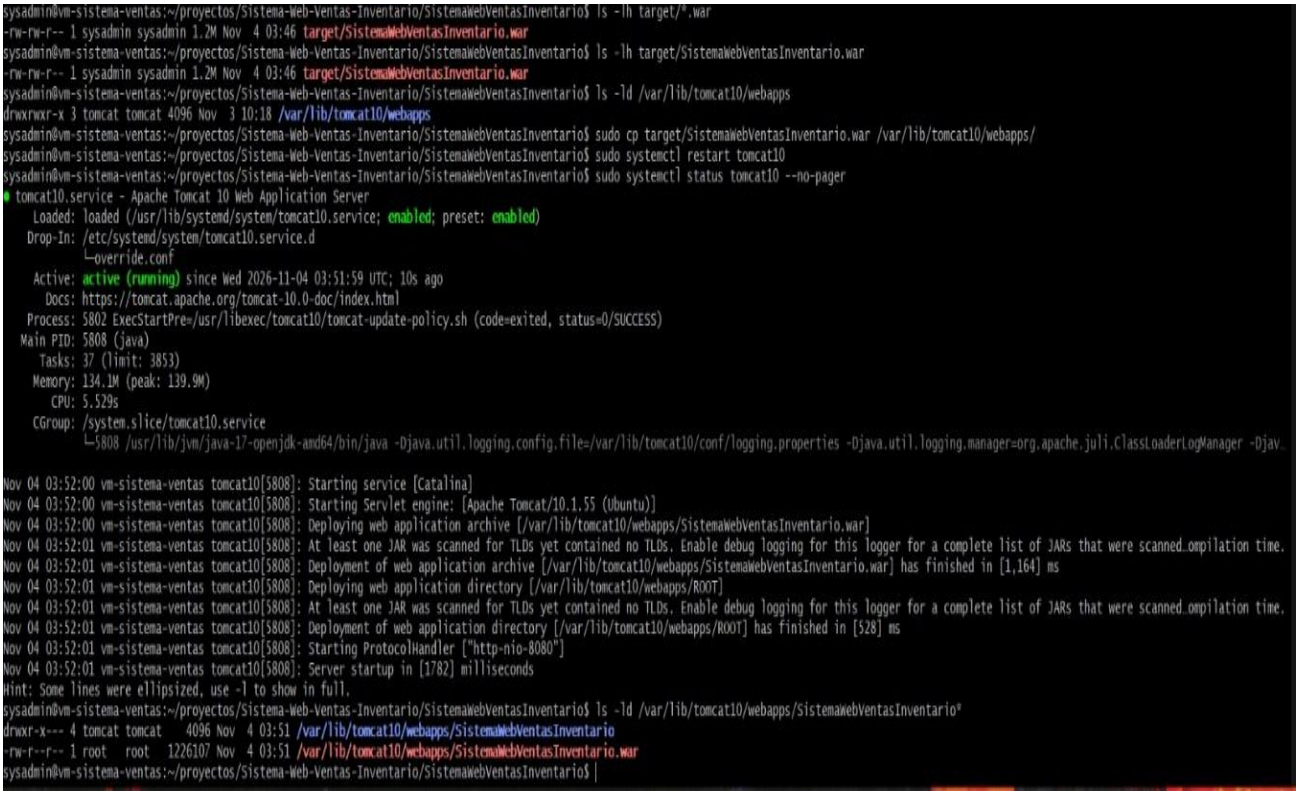


Usuario de PostgreSQL app_inventario. Conexión a sistema_inventario_web con psql y asignación de la contraseña del usuario de aplicación, sin mostrar su valor. Fuente: evidencias de infraestructura Linux, evidencia 14.





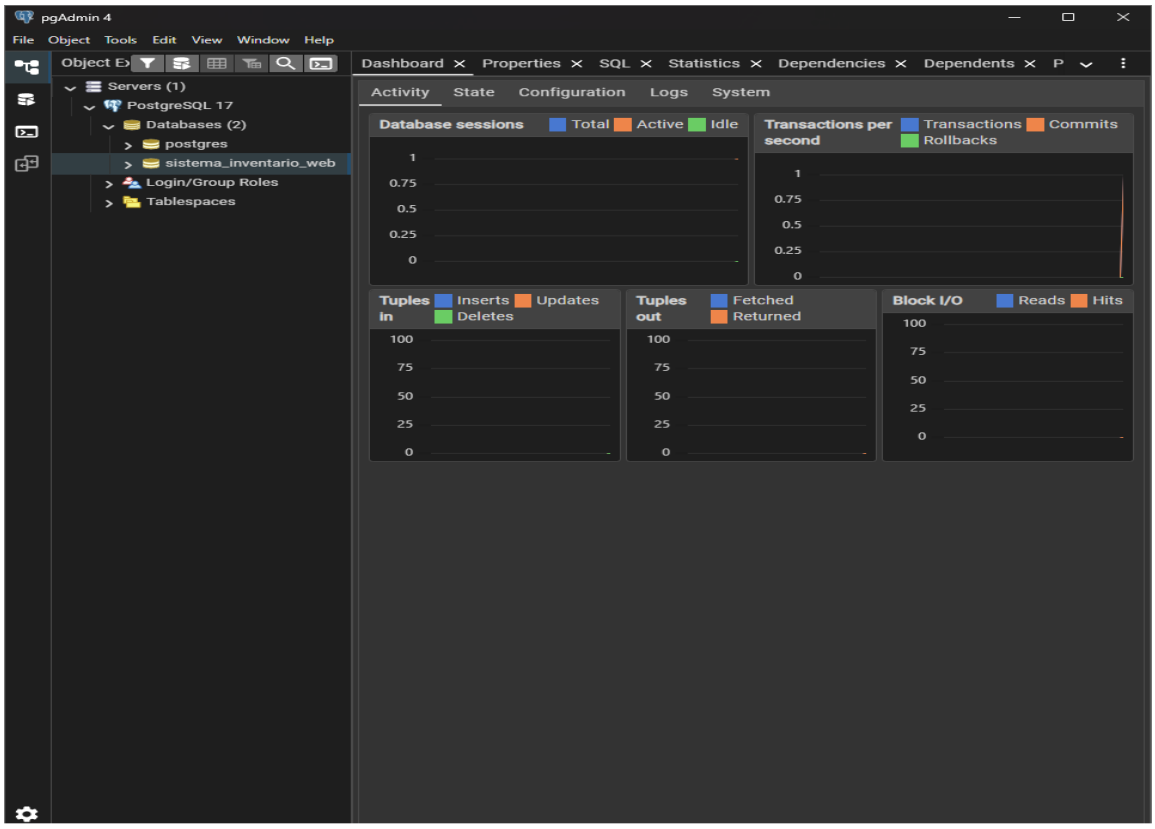
Compilación del WAR mediante Maven. Compilación y empaquetado de SistemaWebVentasInventario.war. Fuente: evidencias de infraestructura Linux, evidencia 16.



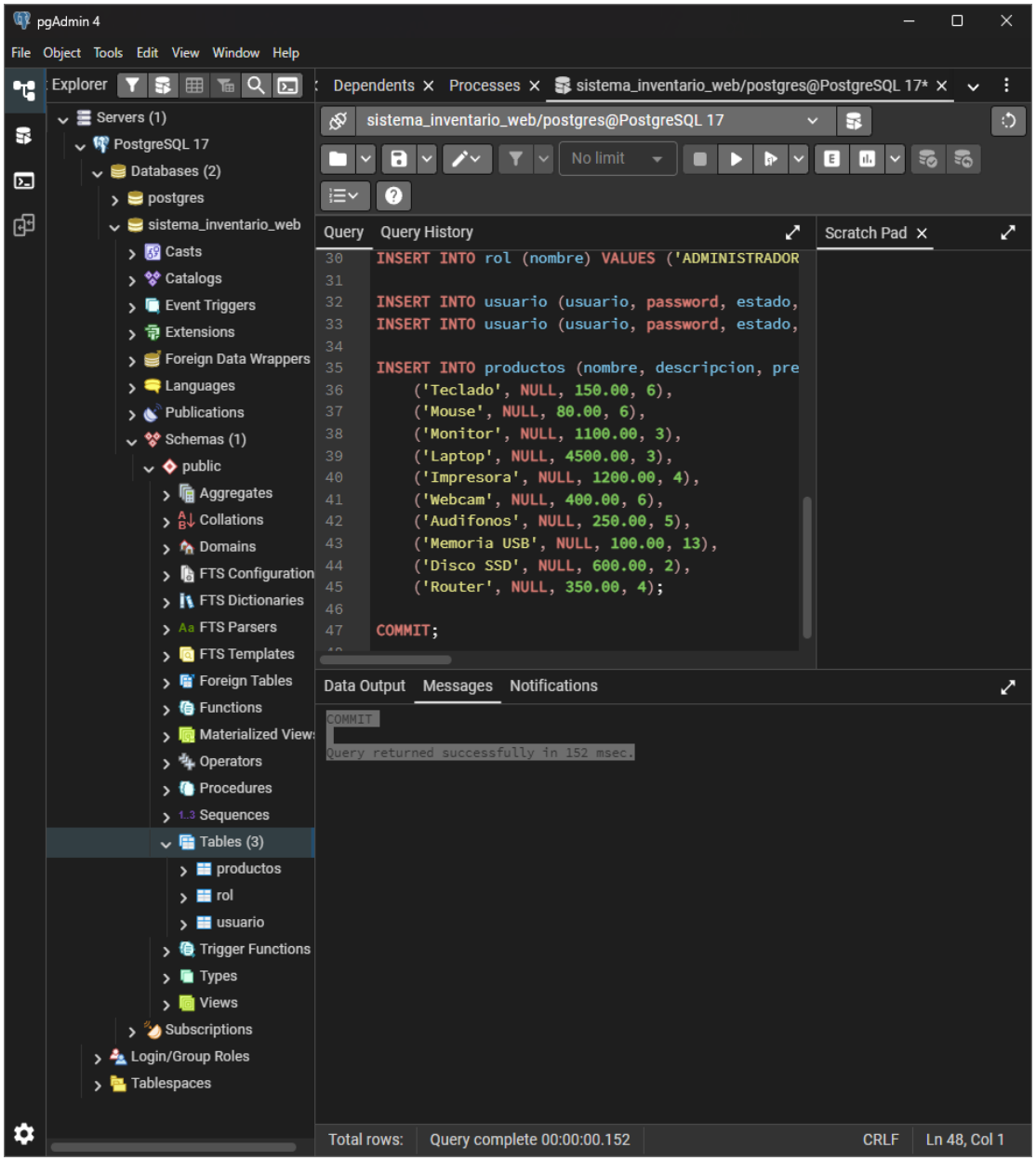
Despliegue del WAR en Apache Tomcat. Copia del WAR a /var/lib/tomcat10/webapps/ y comprobación del arranque en el servidor. Fuente: evidencias de infraestructura Linux, evidencia 17.

13.2 Evidencias del backend y PostgreSQL

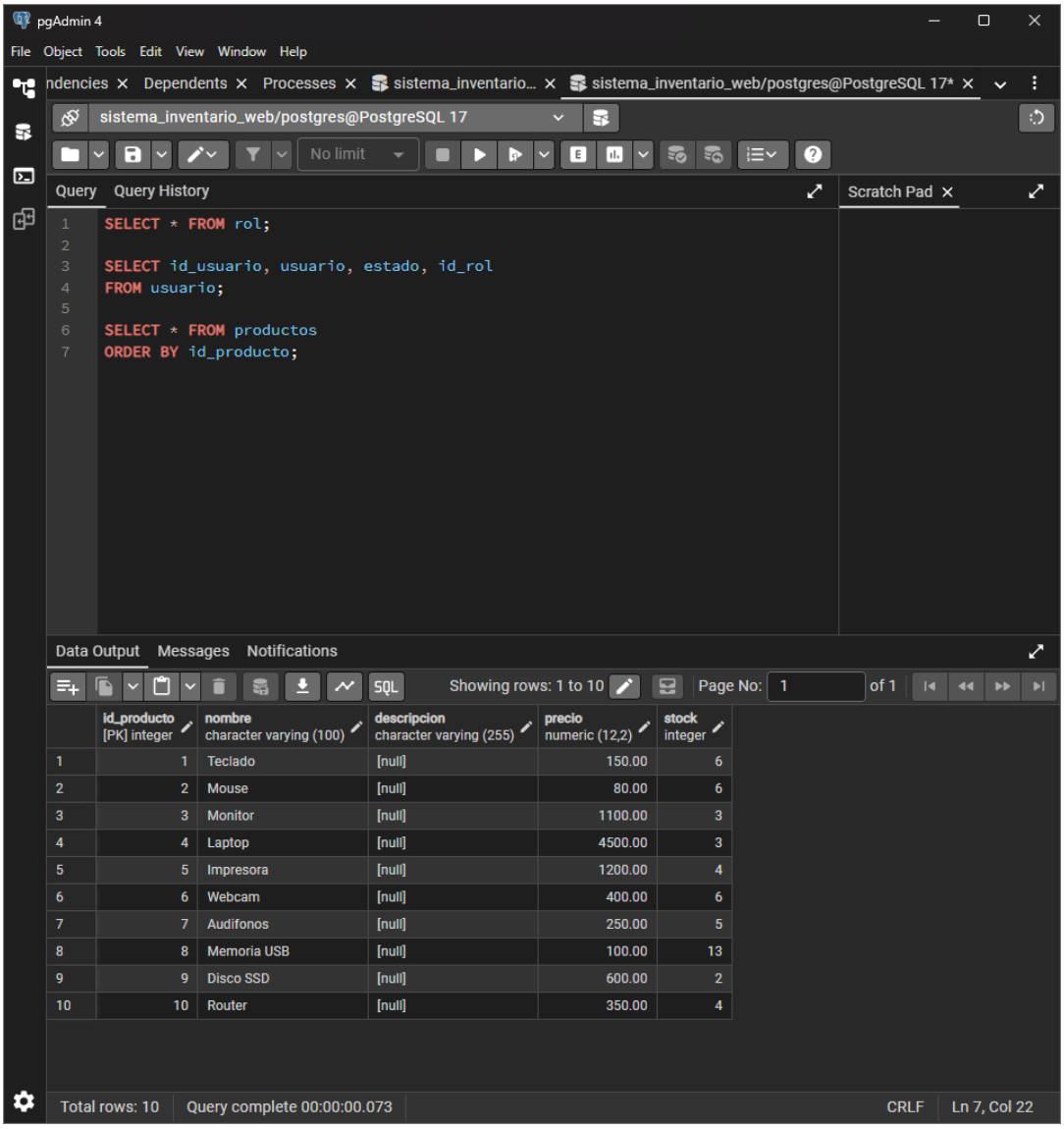
Selección de capturas del desarrollo y las pruebas locales del backend y de PostgreSQL. Representan etapas sucesivas del desarrollo, por lo que algunas pantallas muestran un estado anterior a la versión final; no son capturas del VPS.



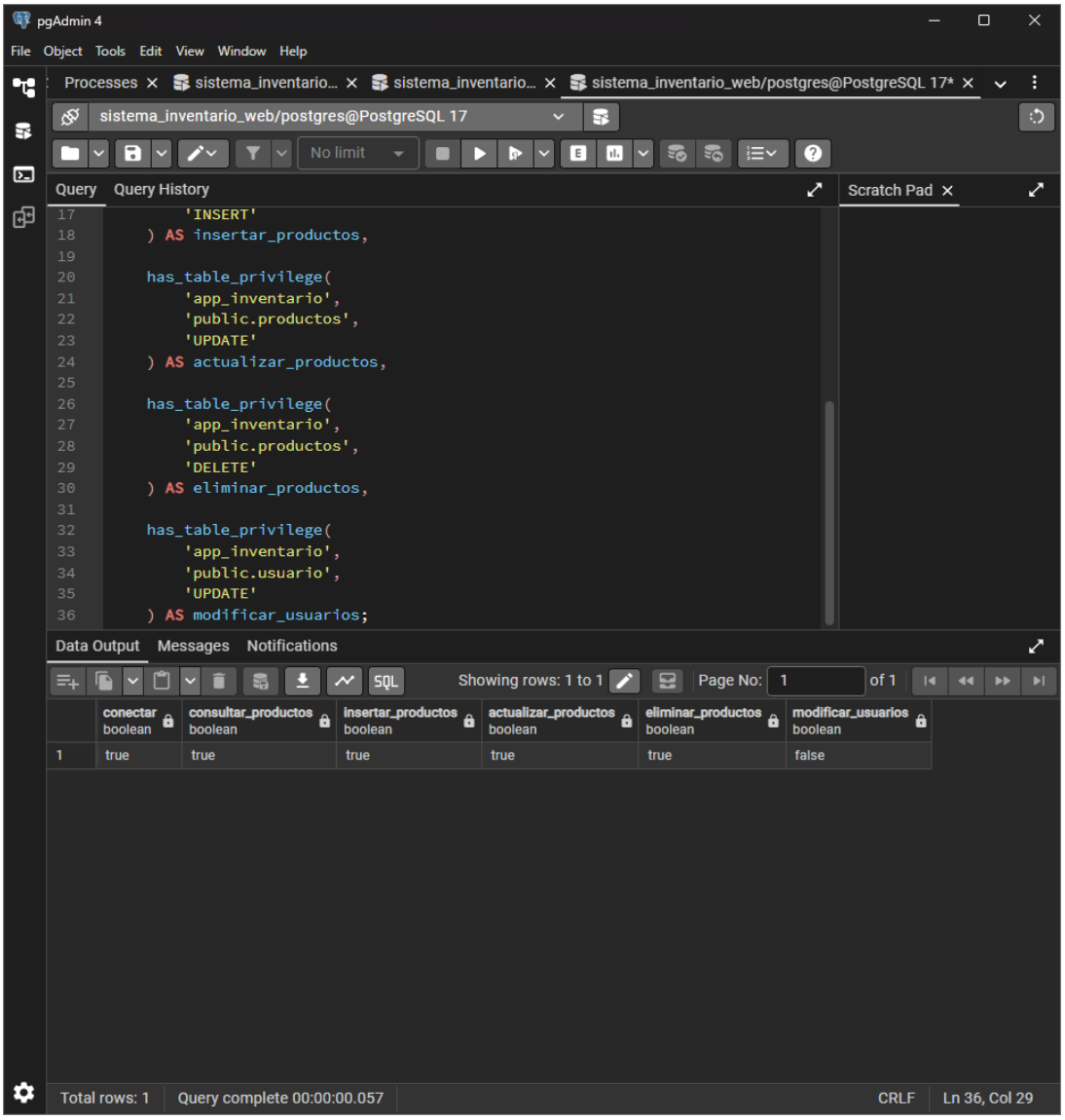
Base de datos sistema_inventario_web en pgAdmin. La base del proyecto aparece en el árbol del servidor PostgreSQL 17 local. Fuente: informe técnico del backend, evidencia E02.



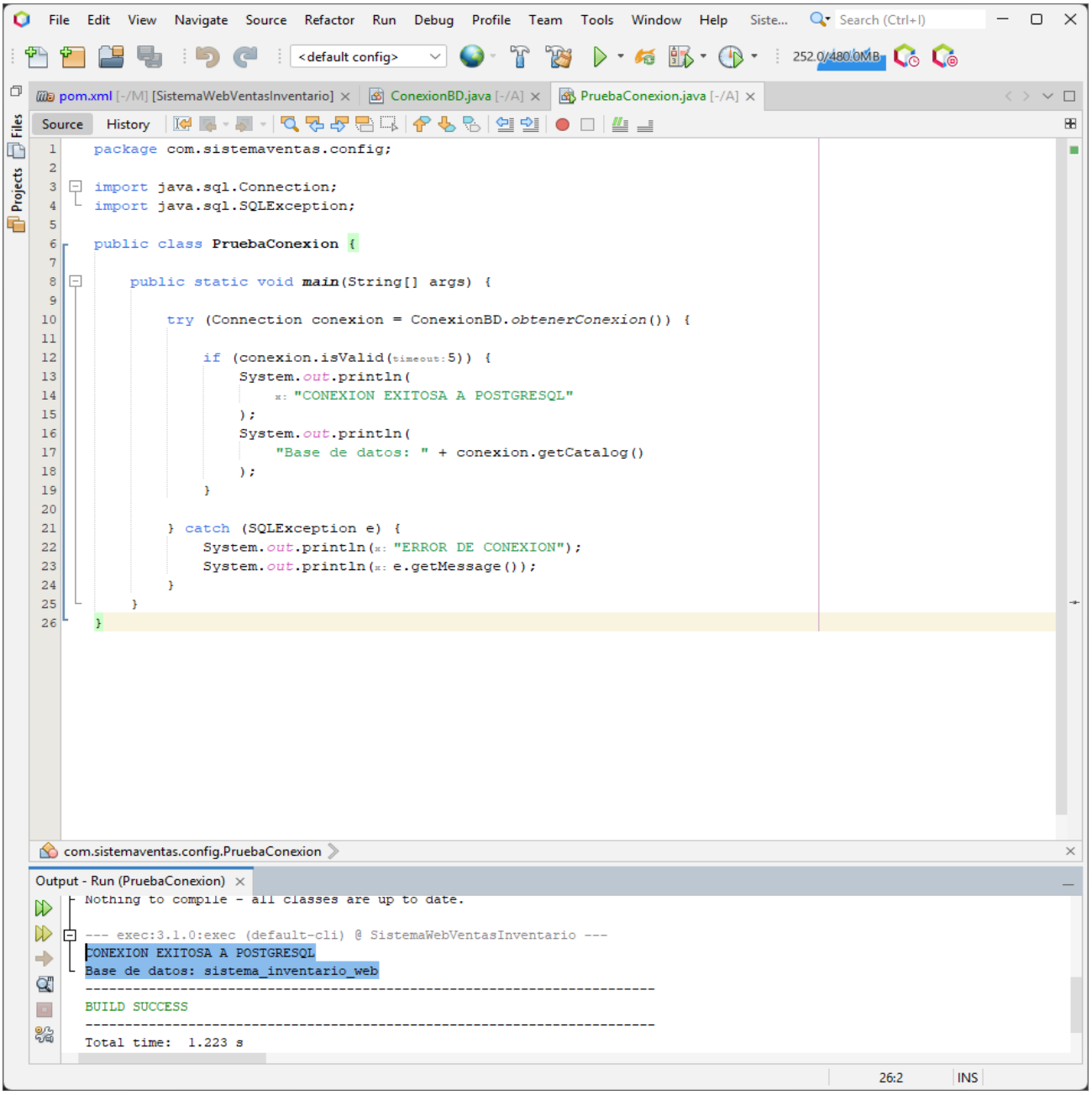
Tablas creadas en el esquema public. Se distinguen las tablas productos, rol y usuario y la ejecución de la carga de datos. Fuente: informe técnico del backend, evidencia E04.



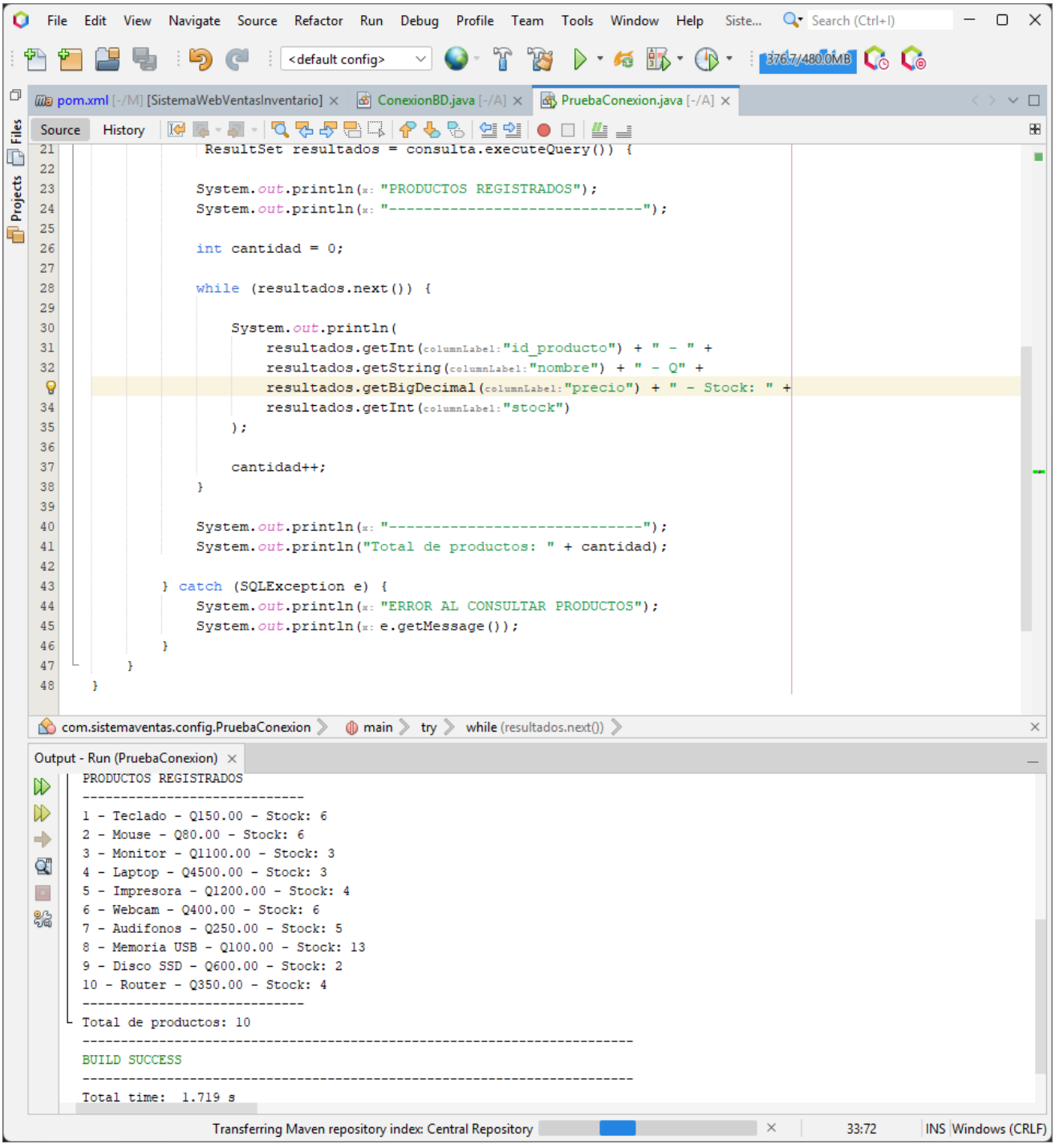
Consulta de productos y resultados. El Query Tool muestra diez productos con identificador, nombre, precio y existencias. Fuente: informe técnico del backend, evidencia E05.



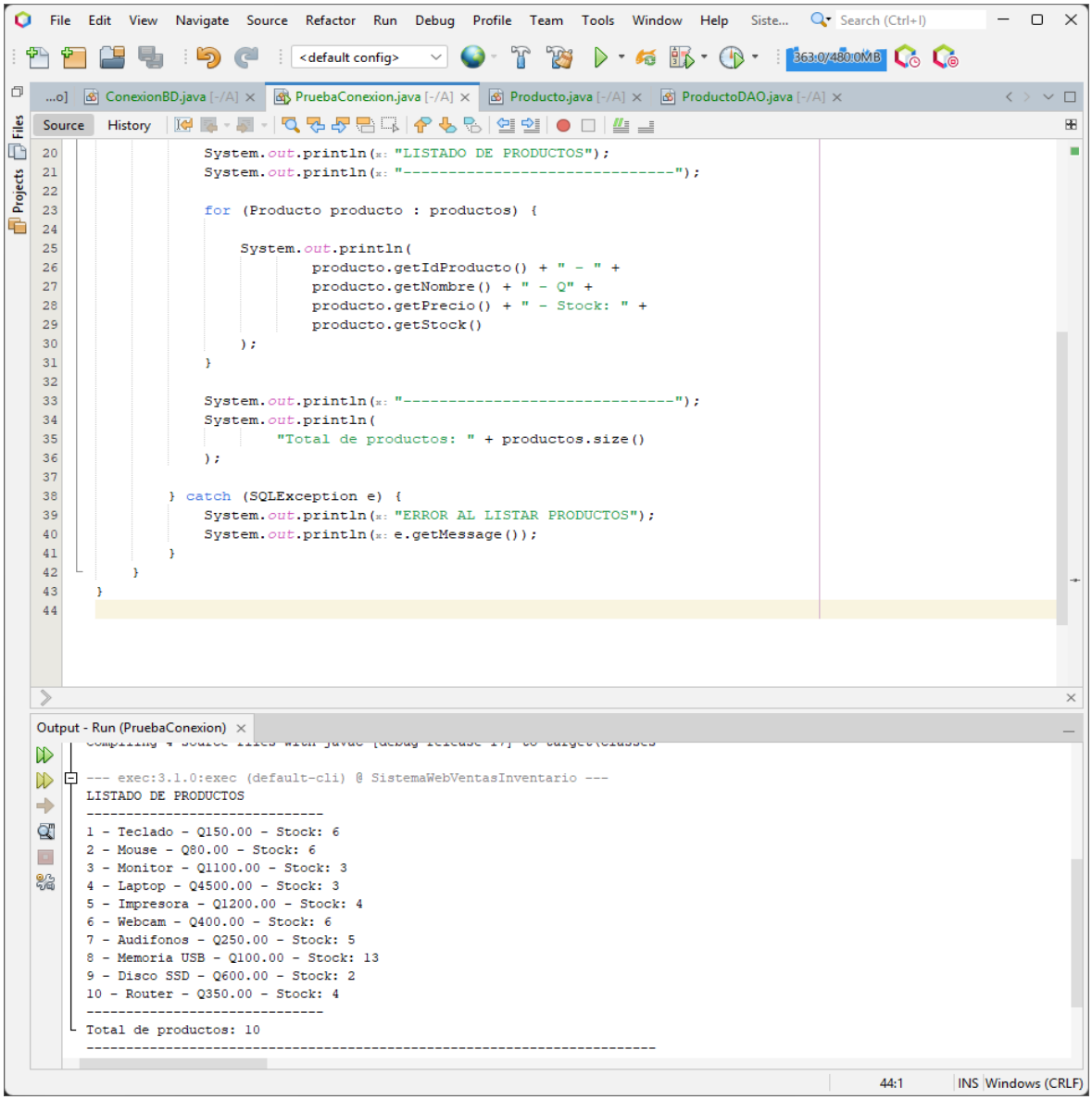
Verificación de privilegios de PostgreSQL. Una consulta de comprobación devuelve los privilegios disponibles sobre los objetos. Fuente: informe técnico del backend, evidencia E07.



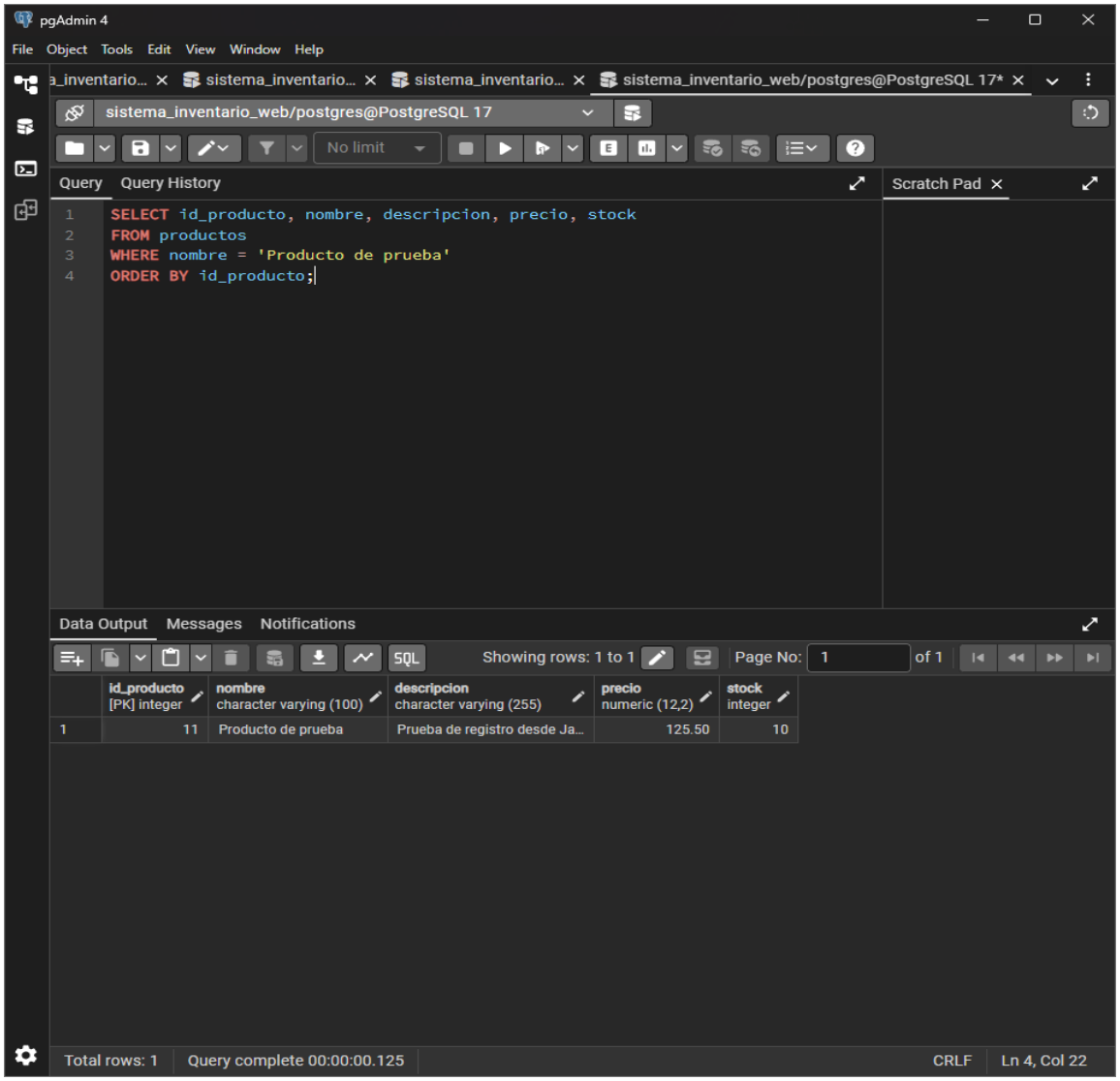
Conexión JDBC desde Java a PostgreSQL. Una prueba en NetBeans confirma la conexión exitosa con la base sistema_inventario_web. Fuente: informe técnico del backend, evidencia E09.



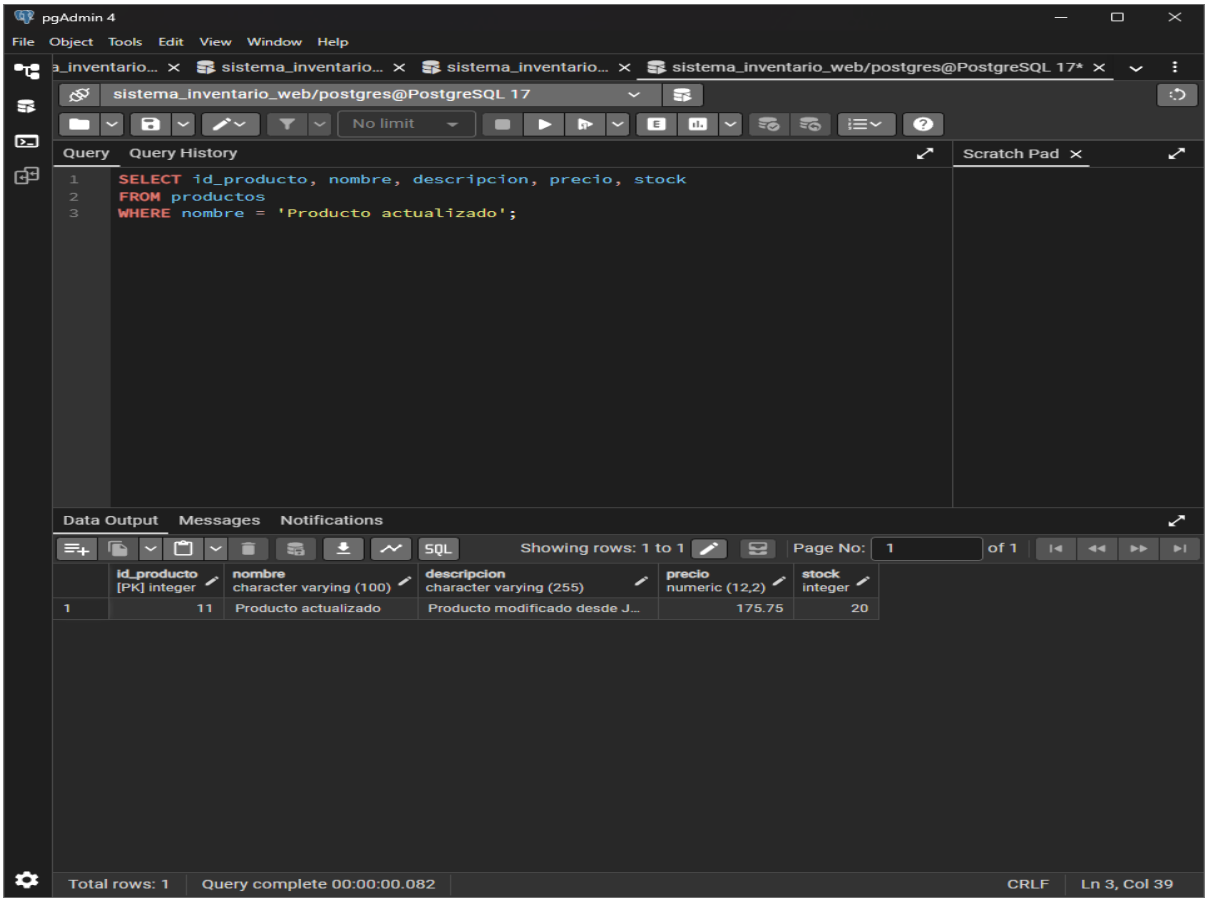
Listado de productos desde la capa DAO. La salida de la prueba Java enumera los productos leídos por ProductoDAO. Fuente: informe técnico del backend, evidencia E10.



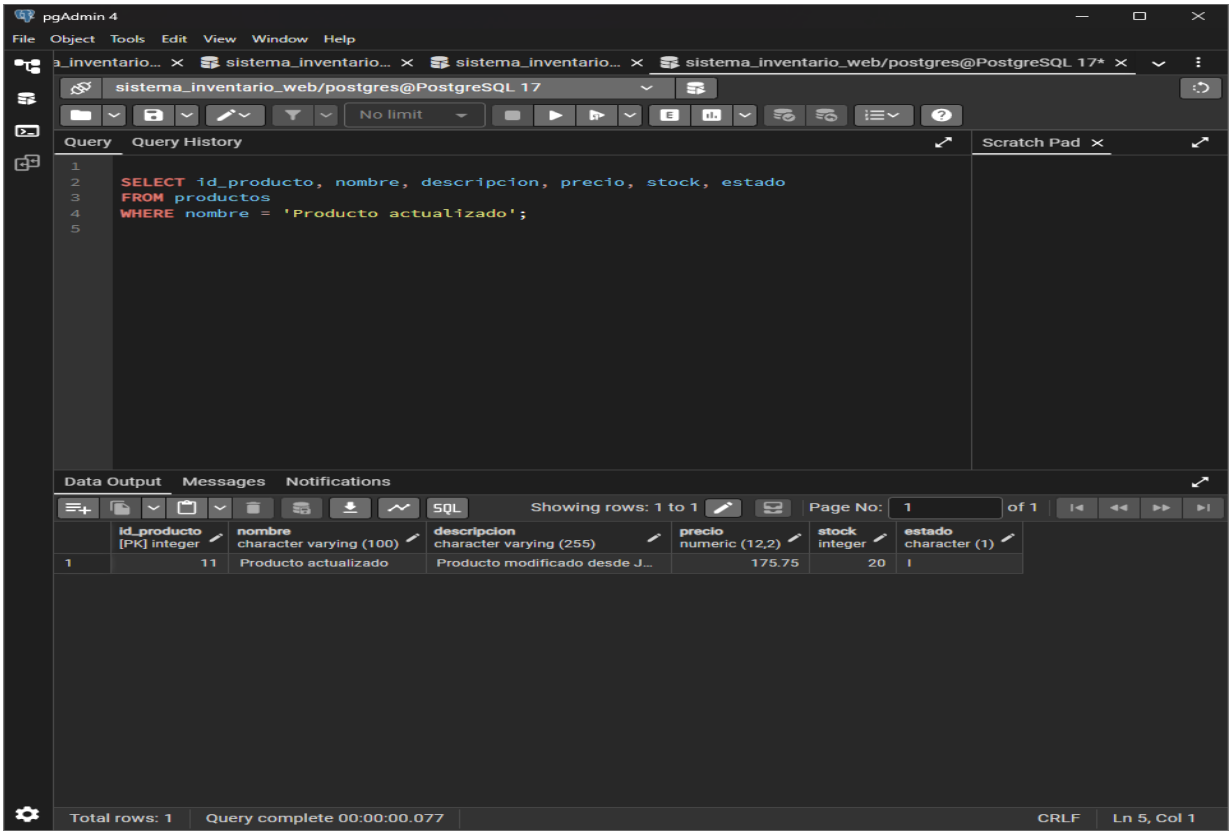
Registro de un producto desde Java. La consola refleja una inserción seguida del listado de productos. Fuente: informe técnico del backend, evidencia E11.



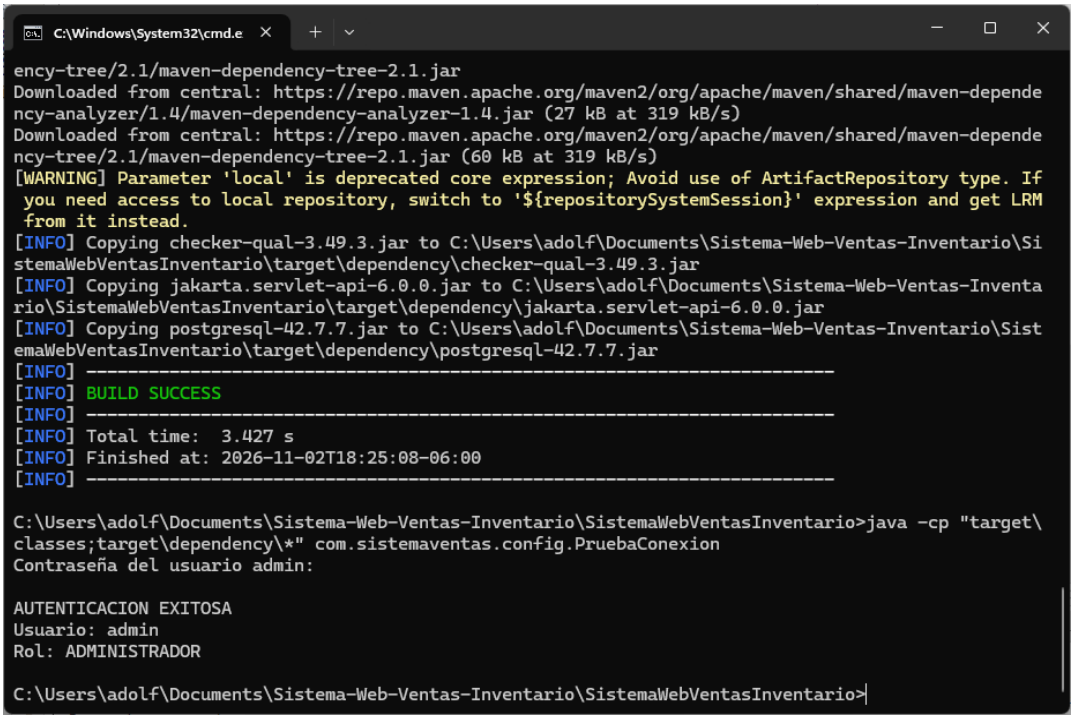
Producto de prueba consultado en PostgreSQL. La consulta muestra el producto insertado desde Java, lo que confirma que llegó a la base. Fuente: informe técnico del backend, evidencia E13.



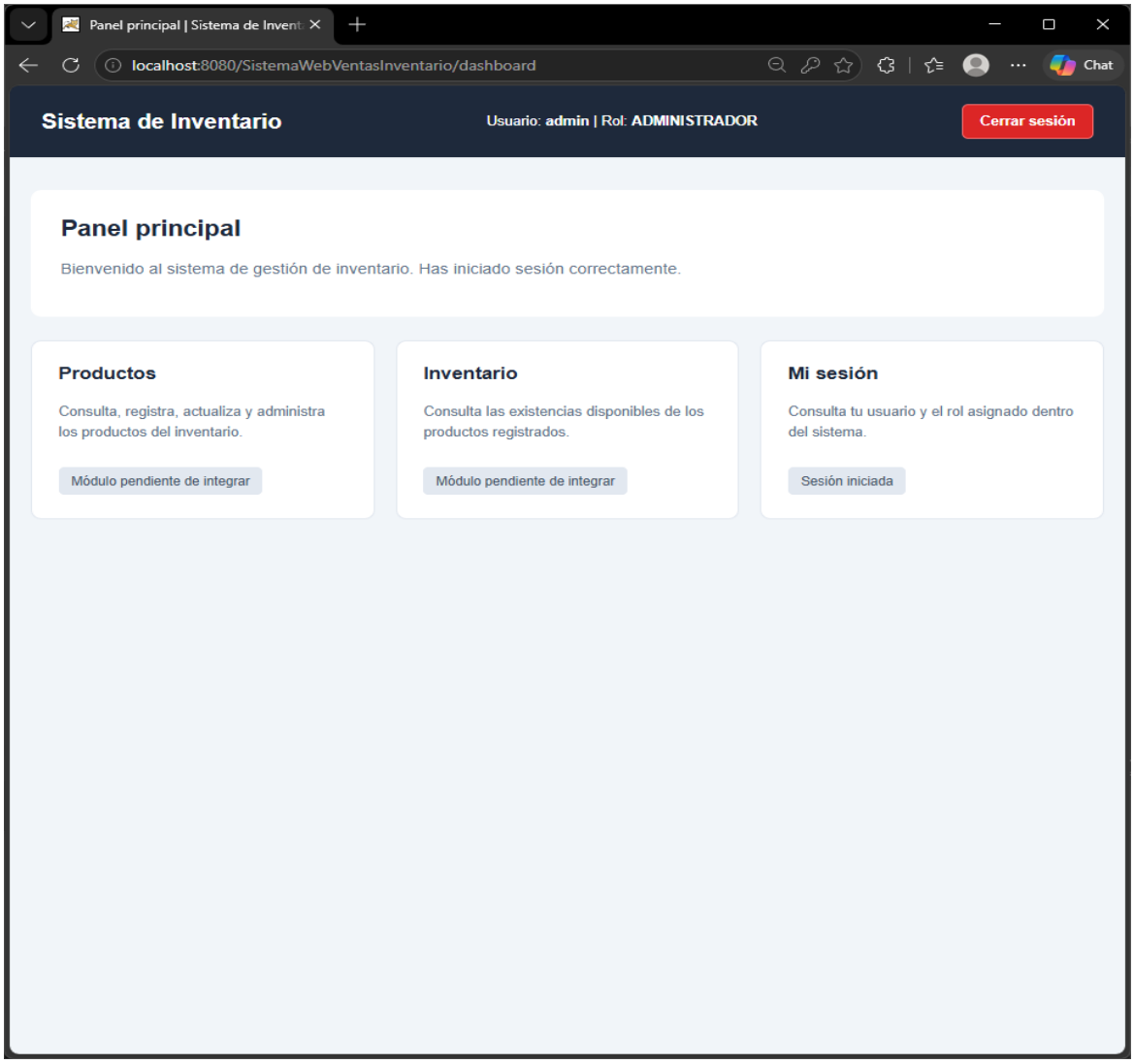
Comprobación SQL de la actualización. pgAdmin muestra los nuevos valores del producto actualizado desde Java. Fuente: informe técnico del backend, evidencia E15.



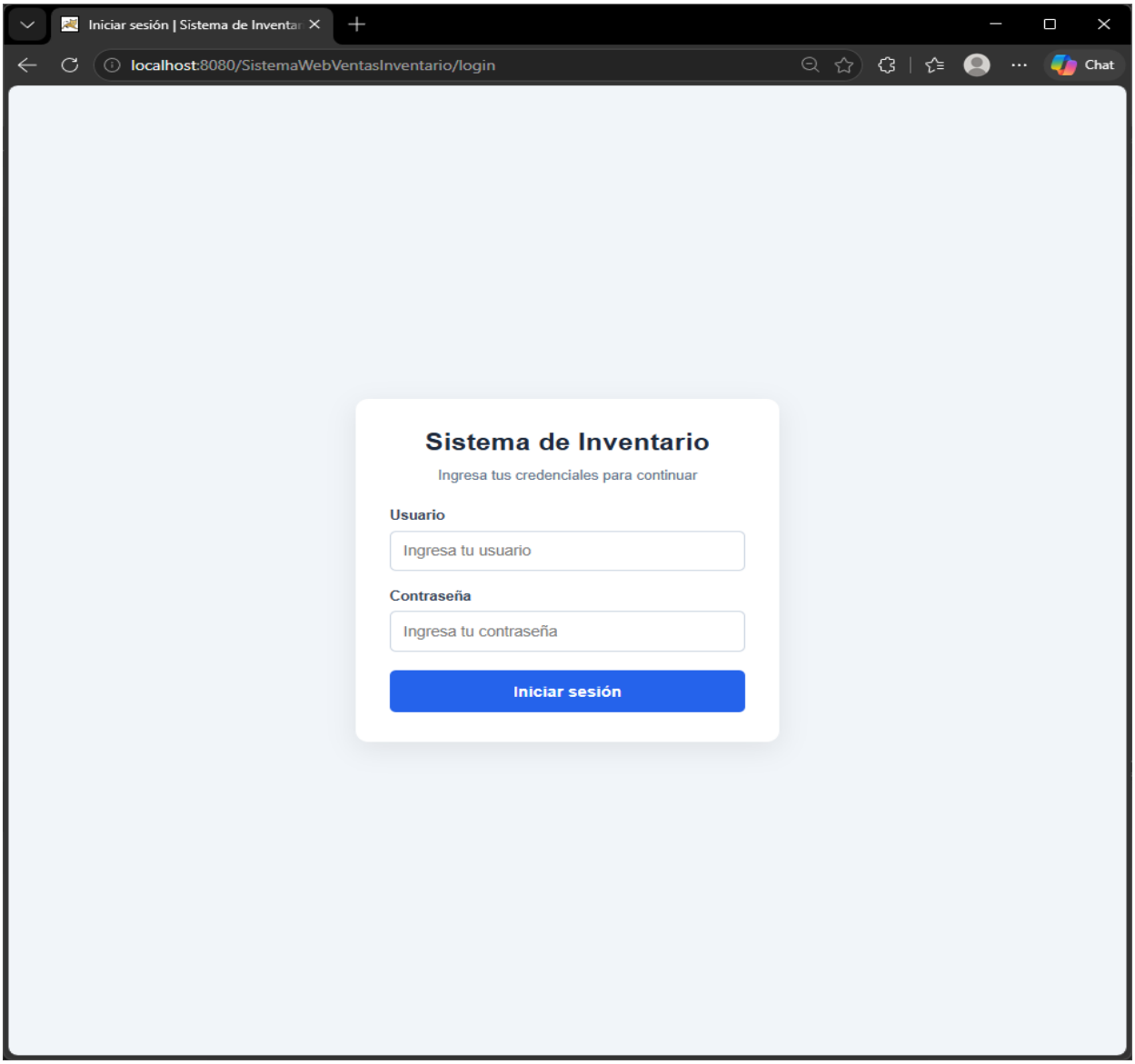
Estado inactivo confirmado mediante SQL. El producto de prueba aparece con estado I, efecto de la desactivación lógica. Fuente: informe técnico del backend, evidencia E18.



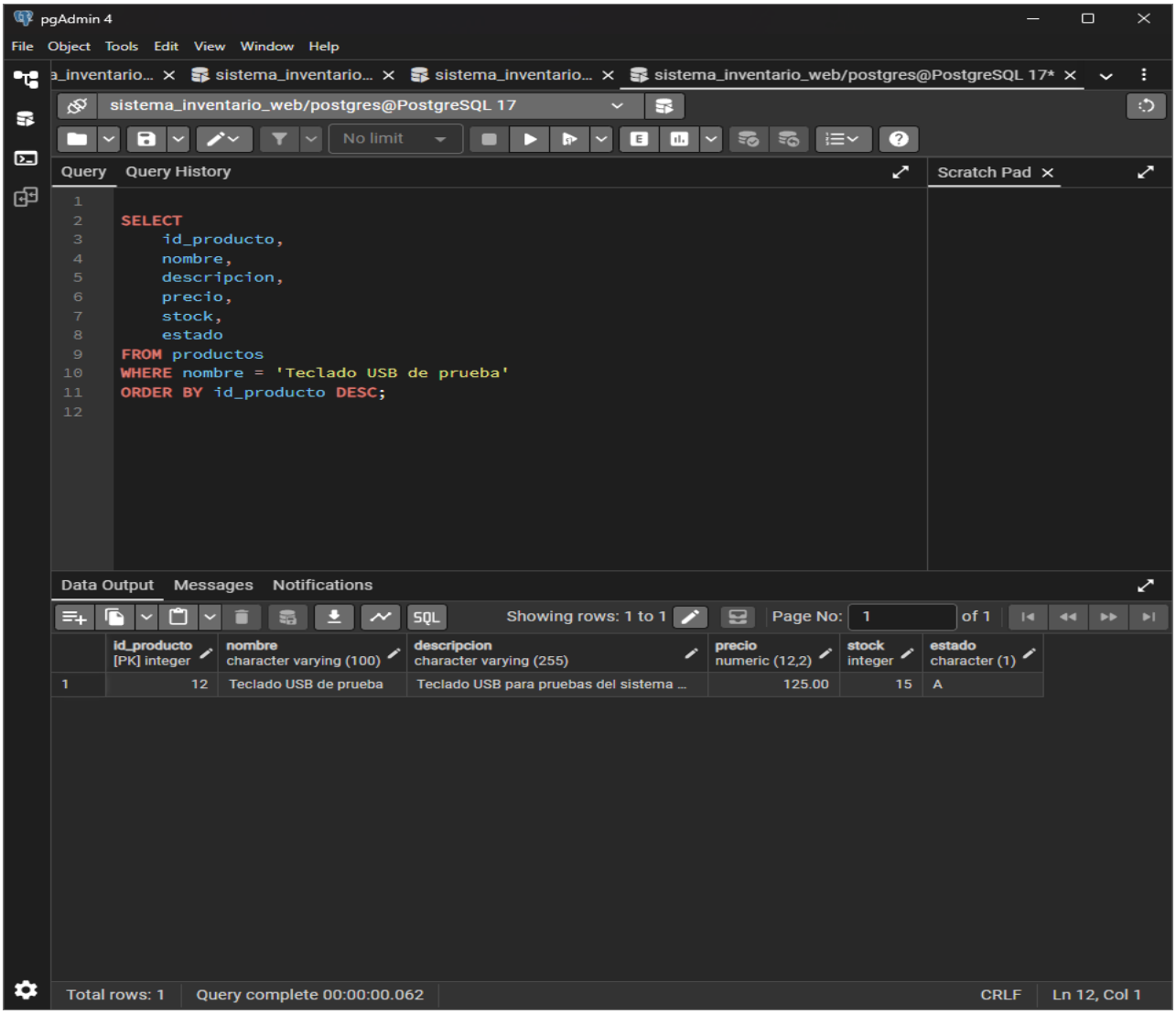
Compilación y autenticación correcta del administrador. La terminal muestra BUILD SUCCESS y AUTENTICACION EXITOSA para admin con rol ADMINISTRADOR. Fuente: informe técnico del backend, evidencia E19.



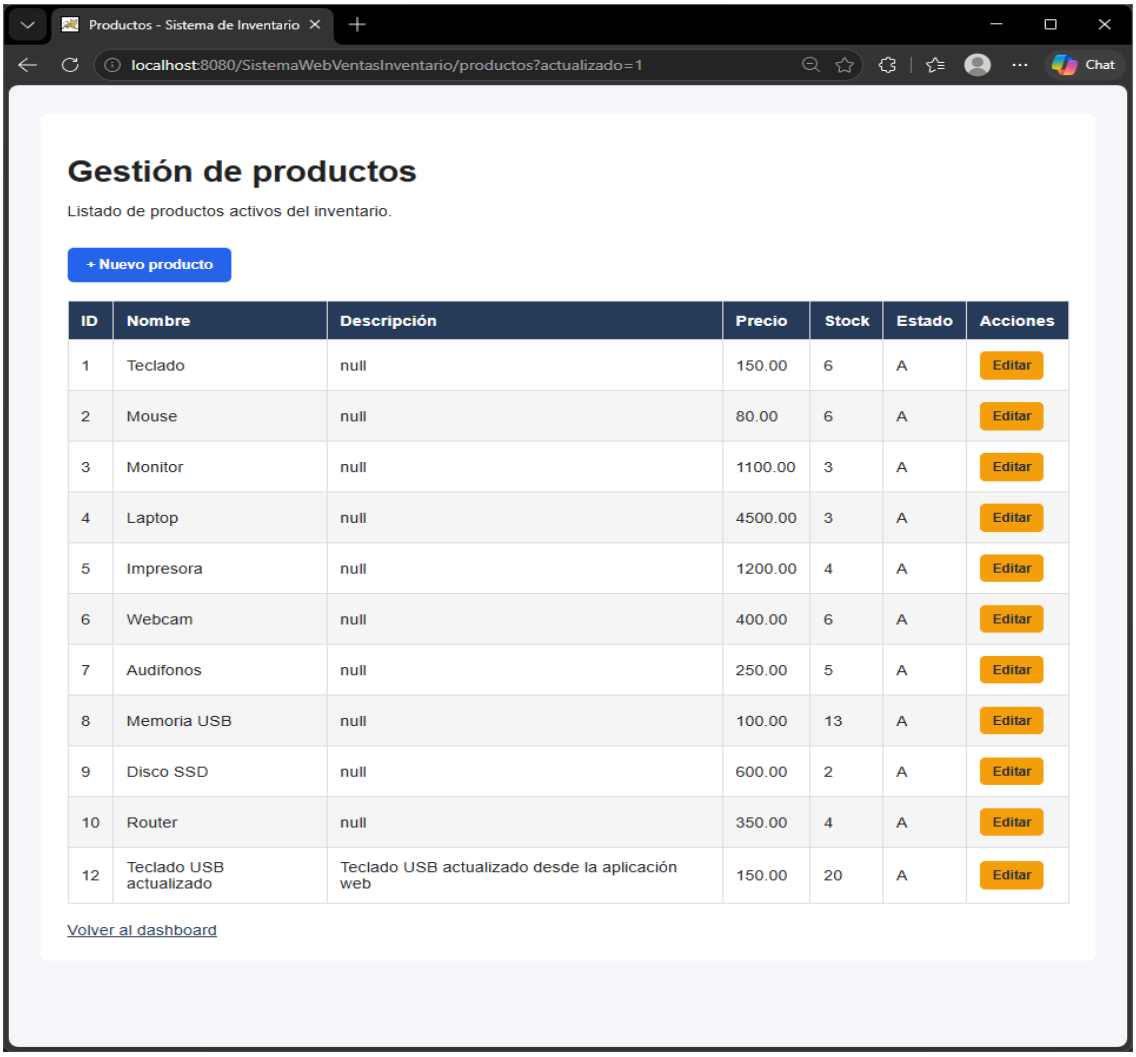
Panel principal con sesión de administrador. Dashboard local con la sesión del usuario admin; versión anterior a los últimos ajustes visuales. Fuente: informe técnico del backend, evidencia E20.



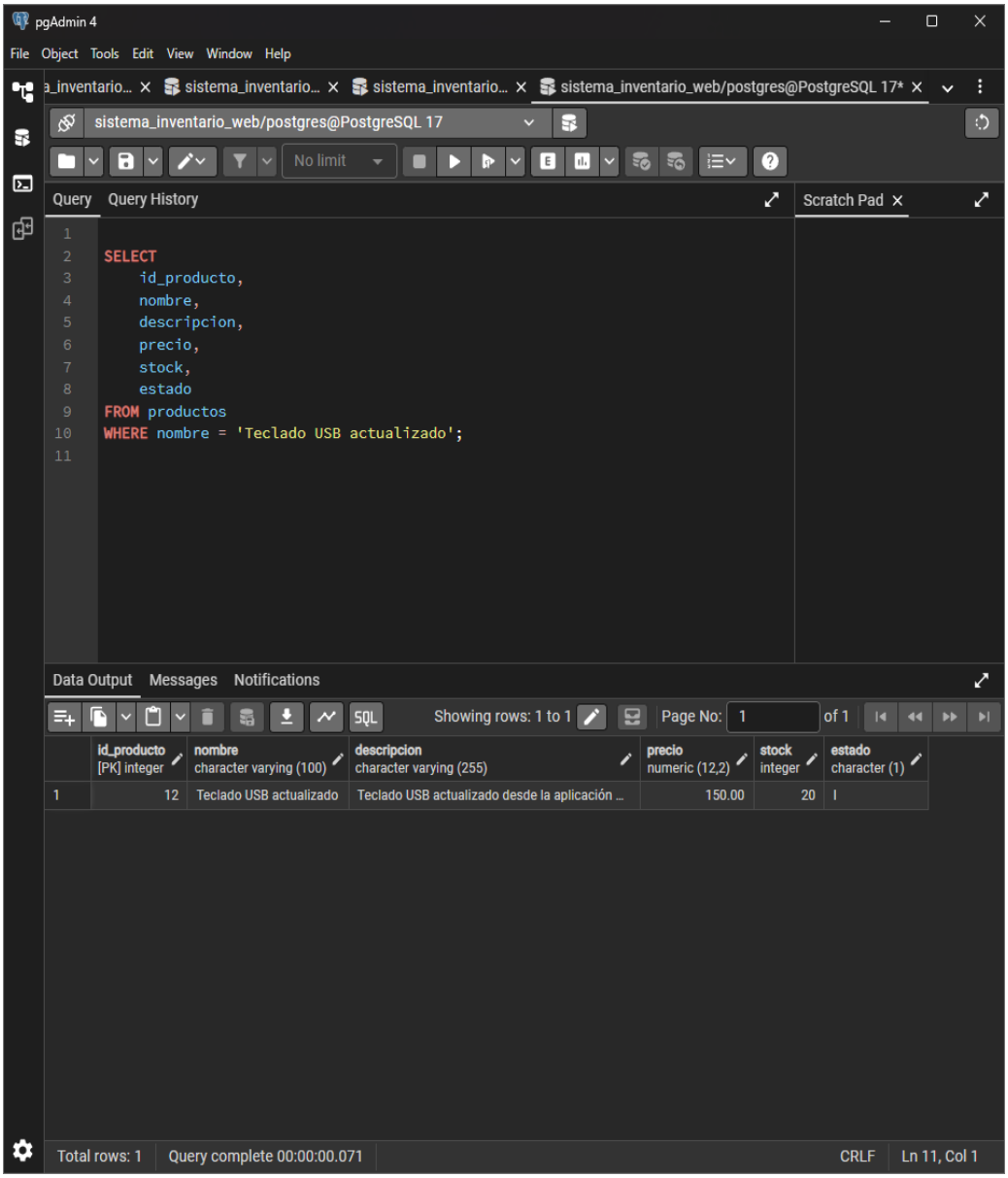
Formulario web de inicio de sesión. Pantalla de login local con campos de usuario y contraseña. Fuente: informe técnico del backend, evidencia E21.



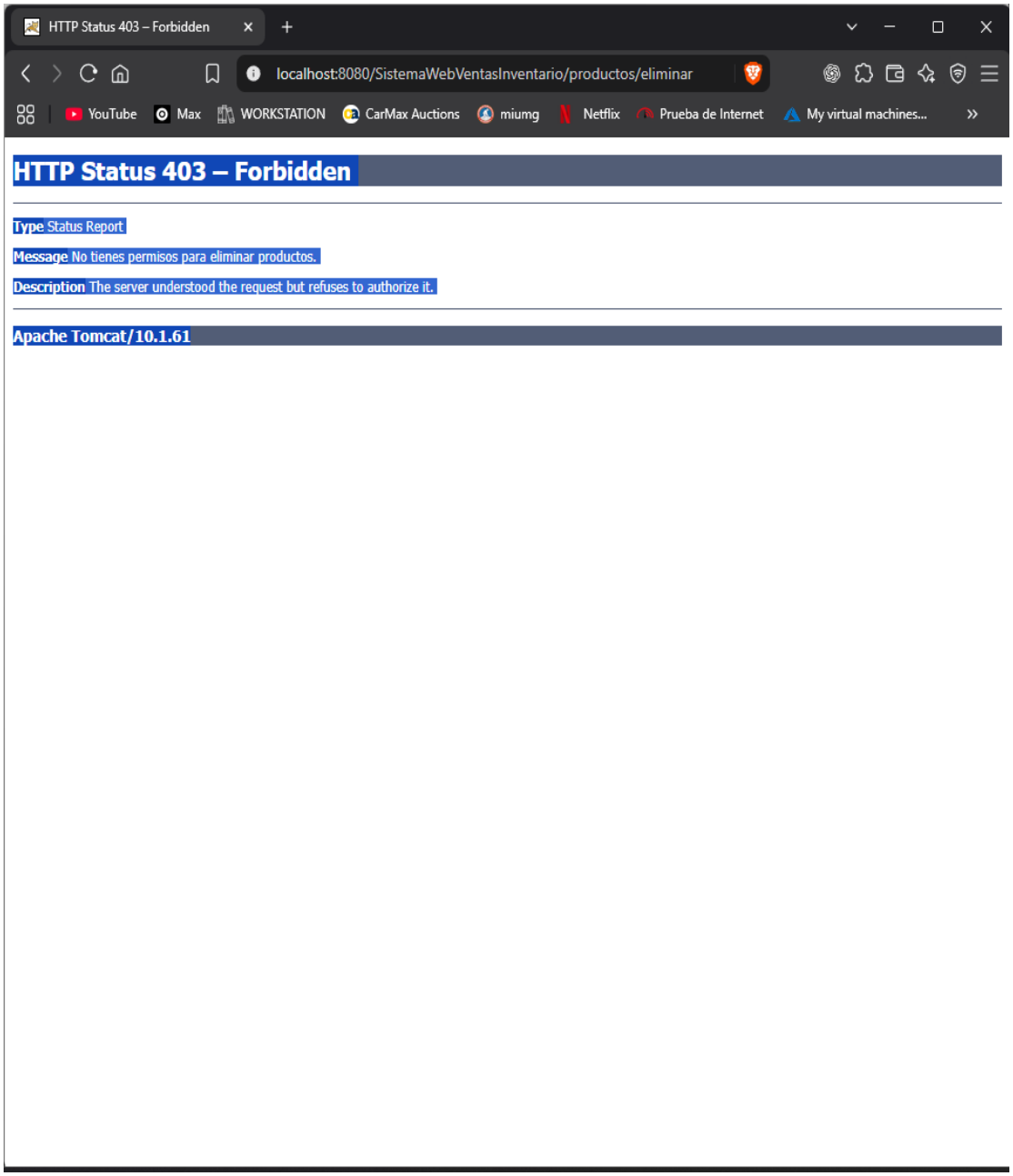
Validación SQL del producto creado desde la web. La consulta localiza el producto añadido desde la interfaz: el registro quedó guardado en la base. Fuente: informe técnico del backend, evidencia E24.



Producto modificado desde la interfaz web. La tabla web refleja los cambios del producto de prueba. Fuente: informe técnico del backend, evidencia E25.



Validación SQL del producto modificado. La fila muestra el nuevo precio y stock, confirmando que los cambios se guardaron. Fuente: informe técnico del backend, evidencia E26.



Control de autorización: respuesta HTTP 403. Tomcat responde 403 – Forbidden en /productos/eliminar con el mensaje «No tienes permisos para eliminar productos». Fuente: informe técnico del backend, evidencia E28.

13.3 Fuentes documentales

«capturas_linux_ordenadas_y_descritas.docx». Evidencias, procedimiento de despliegue y procedimiento de monitoreo del servidor Ubuntu, noviembre de 2026.

«Informe_Completo_Backend_PostgreSQL_Evidencias.pdf». Informe técnico y evidencias de backend y PostgreSQL, 3 de noviembre de 2026.

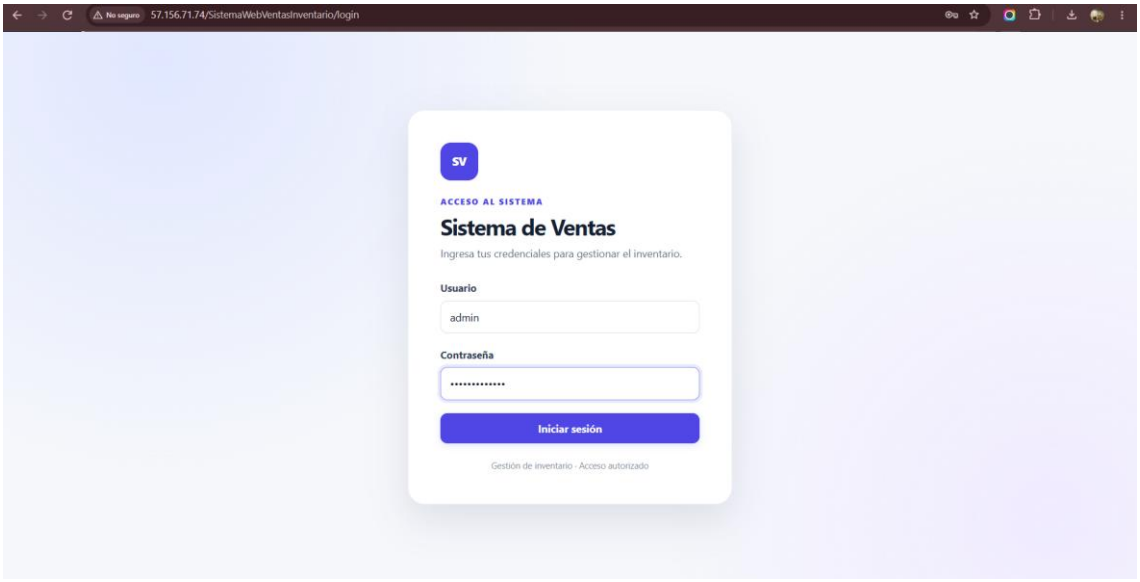
Repositorio del proyecto en GitHub: wallo56/Sistema-Web-Ventas-Inventario.

URL de acceso al sistema: http://57.156.71.74/SistemaWebVentasInventario/login

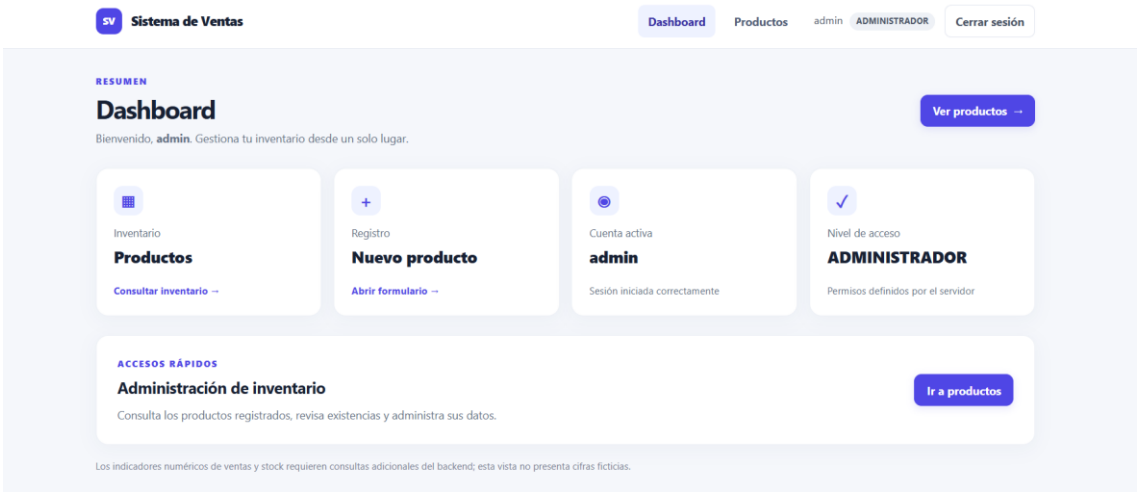
14. Pruebas funcionales: usuario Administrador

Esta sección incorpora las evidencias de las pruebas realizadas desde el navegador con la cuenta admin, cuyo rol aparece identificado como ADMINISTRADOR. Las imágenes muestran el estado de la interfaz antes y después de diversas operaciones de inventario. La persistencia de este tipo de operaciones en PostgreSQL se comprobó en el entorno local mediante las consultas de la sección 13.2.

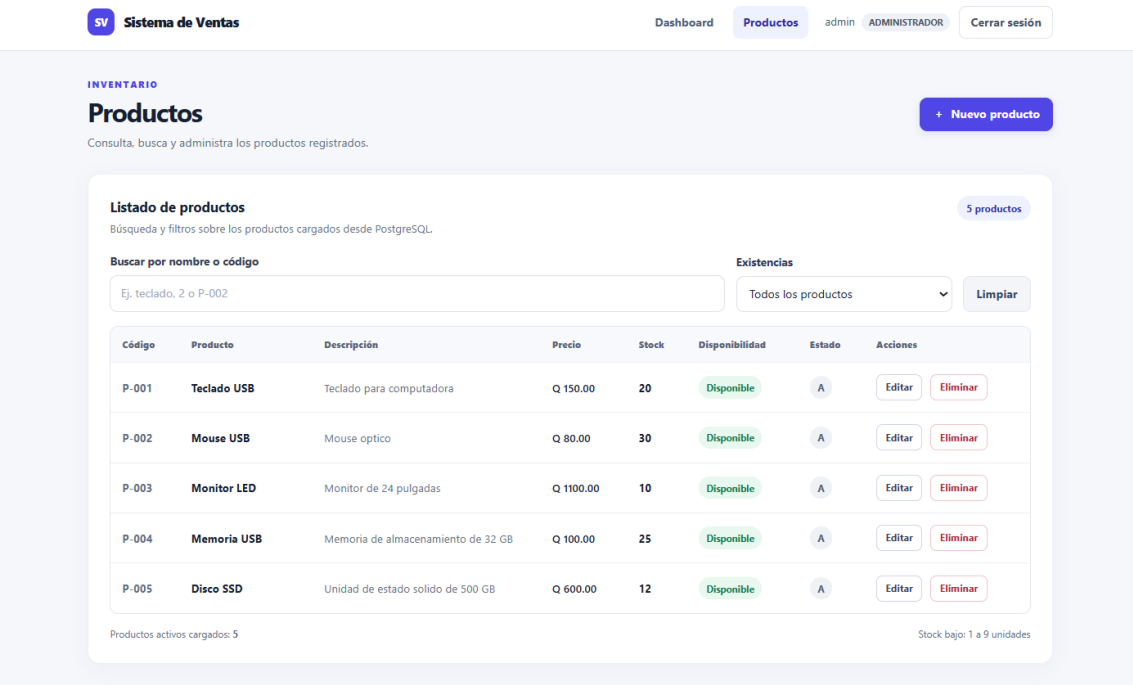
Prueba	Resultado observado	Estado
Inicio de sesión	Se observa el formulario con el usuario admin y la contraseña oculta; una captura posterior muestra el dashboard autenticado.	Aprobada
Dashboard	Se muestra la sesión admin, el rol ADMINISTRADOR y los accesos a productos y nuevo producto.	Aprobada
Consulta de inventario	La lista inicial contiene cinco productos activos con código, precio, existencias y acciones.	Aprobada
Registro de producto	Se presenta el formulario de Memoria RAM, precio Q160 y stock 30; luego figura P-010 en una lista de seis productos.	Aprobada
Edición de producto	Memoria USB P-004 pasa de Q100 y 25 unidades a Q150 y 50 unidades en la interfaz.	Aprobada
Eliminación de producto	Memoria RAM P-010 deja de aparecer y el listado vuelve a cinco productos. No se observa la modalidad de eliminación en la base de datos.	Aprobada
Cierre de sesión	La última captura muestra el formulario de login vacío después de la sesión autenticada.	Aprobada



Formulario de acceso del Administrador. Formulario de autenticación con el usuario admin y la contraseña enmascarada; aún no se observa el resultado del ingreso.

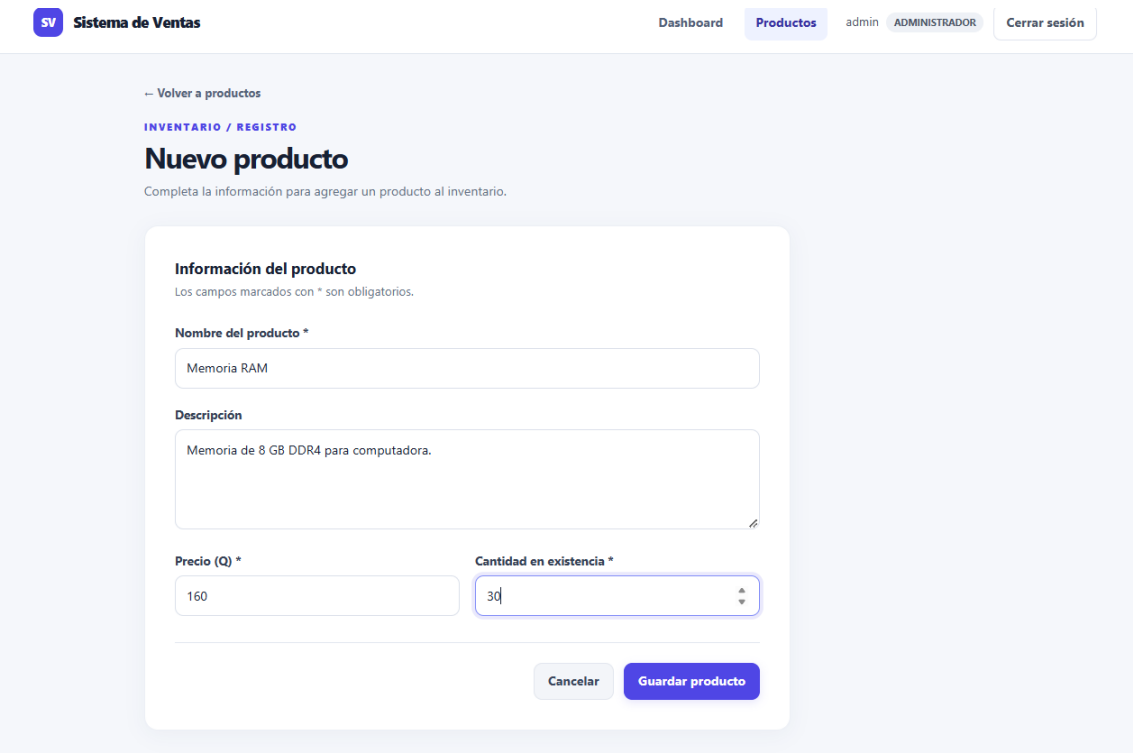


Dashboard del Administrador. Vista de inicio posterior al acceso; identifica al usuario admin con rol ADMINISTRADOR y presenta accesos de inventario.

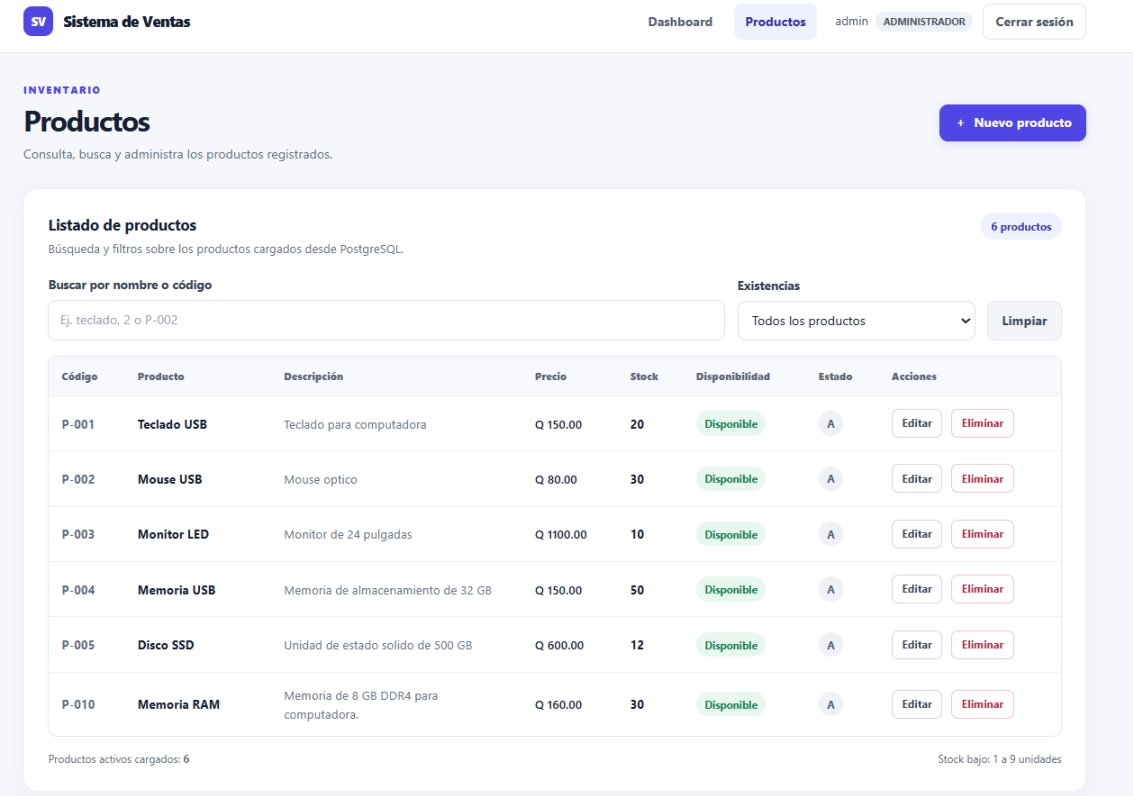


Inventario inicial. Listado de cinco productos con datos de precio, stock, disponibilidad y opciones de edición y eliminación.

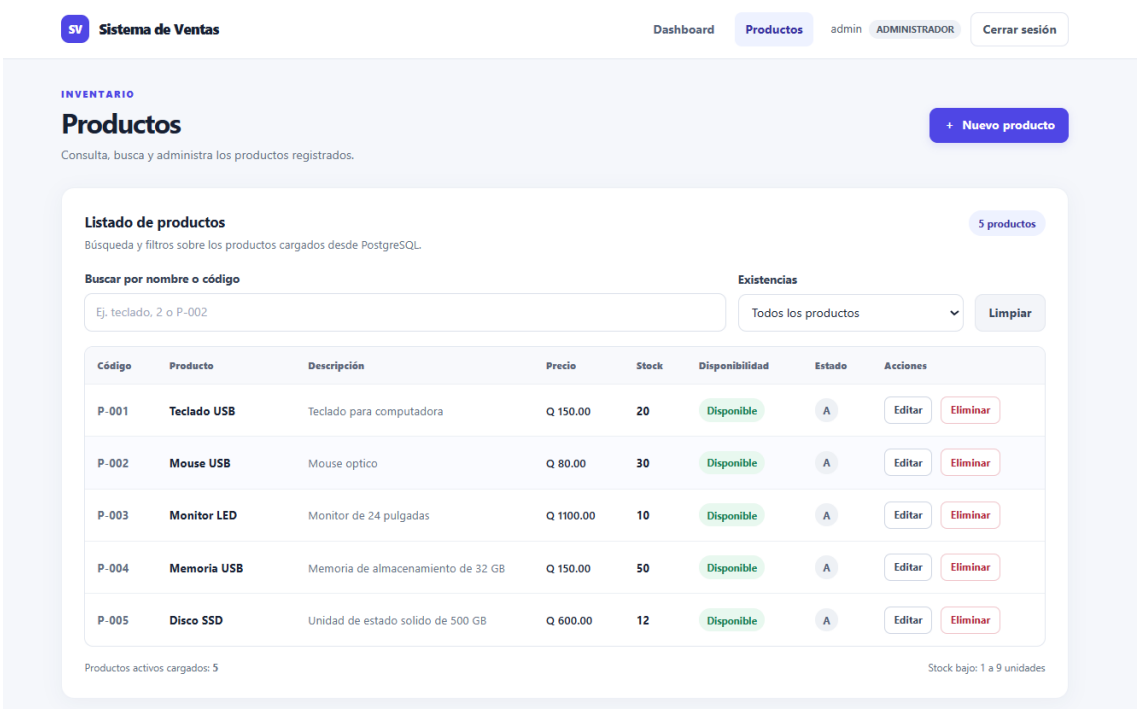
Detalle de producto modificado. Recorte de la fila de Memoria USB P-004 con precio Q150 y stock 50; se compara con el valor anterior Q100 y 25.



Formulario de nuevo producto. Formulario preparado para registrar Memoria RAM con descripción, precio Q160 y 30 unidades.



Producto incorporado al listado. El listado muestra seis productos e incluye Memoria RAM con código P-010, precio Q160 y stock 30.



Inventario tras eliminación. El listado vuelve a cinco productos y ya no muestra Memoria RAM P-010.

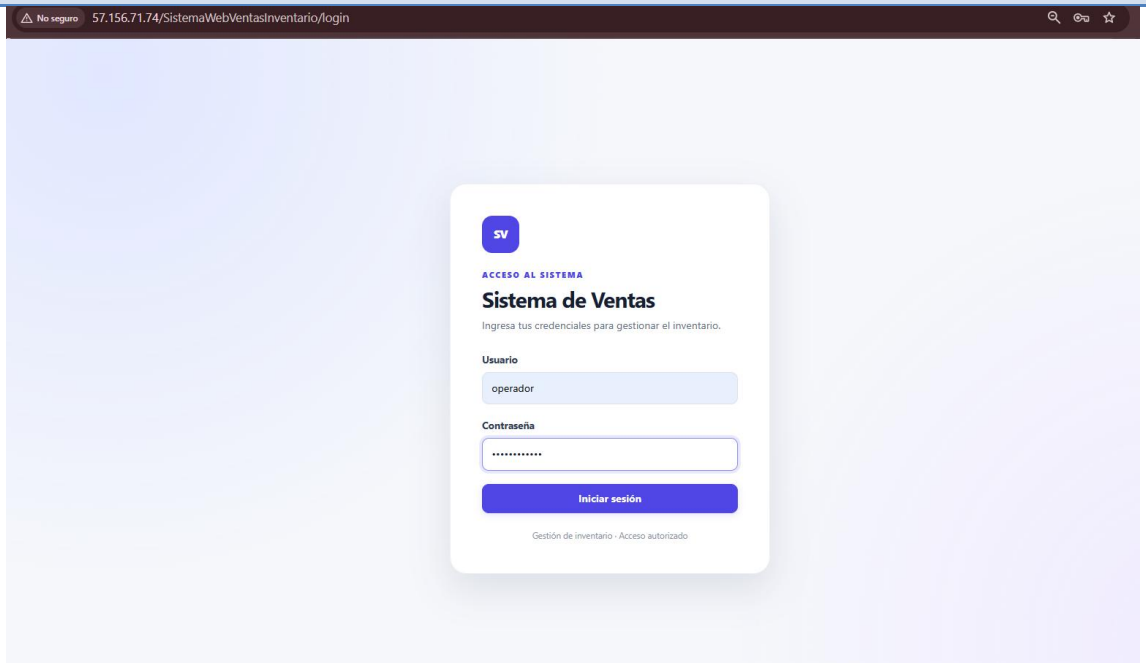


Regreso al inicio de sesión. Se observa nuevamente el formulario de autenticación vacío, compatible con la finalización de la sesión.

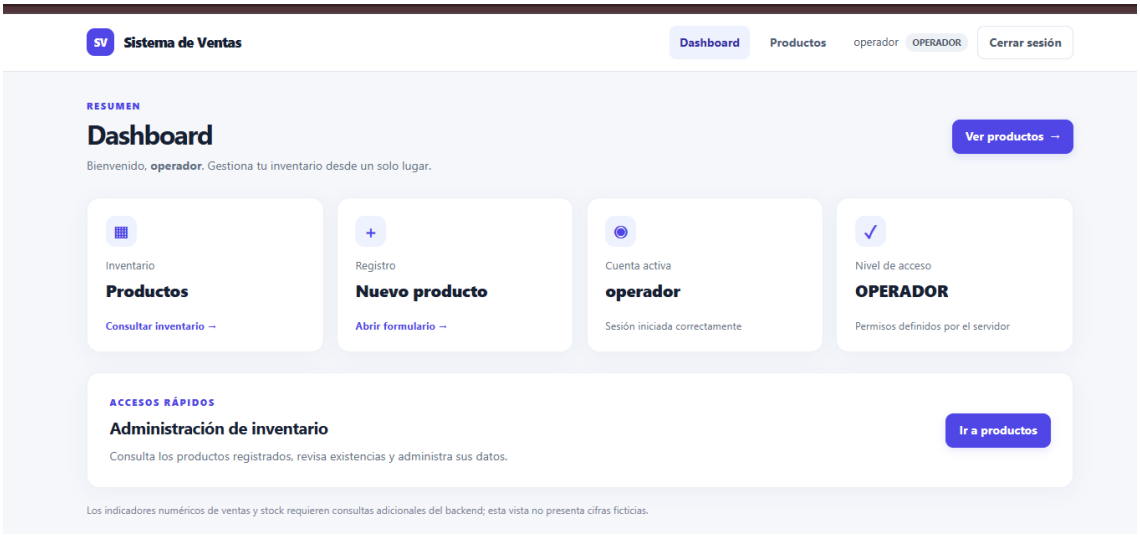
15. Pruebas funcionales: usuario Operador

Se documentan las pruebas realizadas desde el navegador con el usuario operador. El panel muestra el perfil OPERADOR. El listado conserva acciones de edición y registro, pero no muestra el botón Eliminar que sí se observa para Administrador. Esta diferencia visible se registra como control de interfaz por rol, sin atribuirle una respuesta HTTP 403 que no se haya obtenido durante esta prueba.

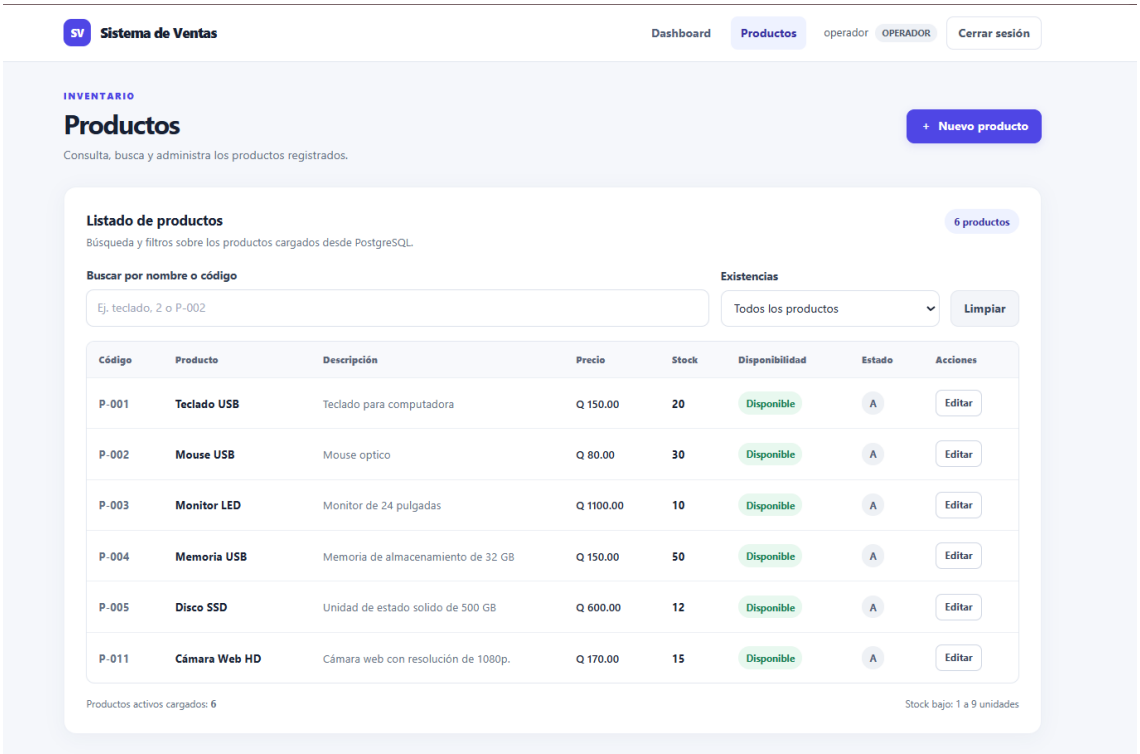
Prueba	Evidencia observada	Resultado
Acceso Operador	Formulario con usuario operador y contraseña enmascarada; dashboard posterior identifica el rol OPERADOR.	Aprobada
Consulta de inventario	Se observa listado de productos con códigos, precios, stock y botón Editar.	Aprobada
Restricción visual de eliminación	En la lista del Operador no aparece el botón Eliminar, a diferencia de la vista del Administrador.	Aprobada
Registro de producto	Se compartió formulario de Cámara Web HD y una lista posterior de seis productos que incluye P-011.	Aprobada
Cierre de sesión	Se compartió la pantalla de acceso vacía al terminar.	Aprobada



Inicio de sesión del Operador. Formulario de autenticación con la cuenta operador y contraseña oculta.



Dashboard del Operador. Panel de inicio que identifica la cuenta y el nivel de acceso OPERADOR.



Inventario con perfil Operador. Listado de productos; las acciones visibles incluyen Editar, sin opción Eliminar. Esta diferencia sustenta la restricción visual por rol.

Las capturas adicionales del Operador muestran el registro de Cámara Web HD (Q170, 15 unidades), el cambio del inventario de cinco a seis productos y el regreso al formulario de inicio. Se incluyen como resultados documentados por la secuencia aportada; su confirmación directamente en PostgreSQL corresponde a una prueba diferente.

16. Evidencias del frontend y scripts PostgreSQL

El frontend integra pantalla de autenticación, dashboard, catálogo de productos, formularios de alta y edición, y cierre de sesión. La interfaz distingue dos perfiles: ADMINISTRADOR y OPERADOR; el primero dispone de la acción Eliminar y el segundo no. Las pruebas del Administrador se desarrollan en la sección 14 y las del Operador en la sección 15.

16.1 Descripción de pantallas del frontend

Las pantallas principales de la interfaz son las siguientes:

Pantalla de inicio de sesión

Formulario de autenticación con campos de usuario y contraseña y botón «Iniciar sesión».

Panel principal (Dashboard)

Panel posterior al inicio de sesión, con usuario autenticado, rol, navegación y botón «Cerrar sesión».

Registro de productos

Formulario de alta con nombre, descripción, precio y cantidad en existencia.

Edición de productos

Formulario de actualización de los datos de un producto registrado.

Eliminación de productos

Vista de administrador con el botón «Eliminar» y cuadro de confirmación previo a la operación.

Cierre de sesión

Retorno a la pantalla de autenticación después de cerrar sesión.

16.2 Archivos SQL del repositorio

En GitHub, la carpeta database/ contiene los archivos schema.sql, datos_prueba.sql y permisos.sql. Su contenido se resume en la siguiente tabla y en las figuras posteriores. Los anexos presentan evidencias de estos archivos, no su transcripción completa. Para consultar el código íntegro, se debe acceder a los tres archivos originales del repositorio privado, sin incluir credenciales.

Archivo	Contenido identificado
schema.sql	Creación de rol, usuario y productos; claves primarias, relación usuario–rol y restricciones de estado, precio y stock.
datos_prueba.sql	Inserción de los roles ADMINISTRADOR y OPERADOR, cinco productos de ejemplo y ajuste de secuencia de roles.
permisos.sql	Concesión a app_inventario de acceso al esquema, operaciones CRUD sobre productos, lectura de rol/usuario y uso de secuencias.

```sql
-- ============================================
-- SISTEMA WEB DE INVENTARIO
-- DATOS DE PRUEBA - POSTGRESQL
-- ============================================

-- Roles utilizados por el backend
INSERT INTO public.rol (id_rol, nombre)
VALUES
    (1, 'ADMINISTRADOR'),
    (2, 'OPERADOR');

-- Productos de ejemplo
INSERT INTO public.productos
    (nombre, descripcion, precio, stock, estado)
VALUES
    ('Teclado USB', 'Teclado para computadora', 150.00, 20, 'A'),
    ('Mouse USB', 'Mouse optico', 80.00, 30, 'A'),
    ('Monitor LED', 'Monitor de 24 pulgadas', 1100.00, 10, 'A'),
    ('Memoria USB', 'Memoria de almacenamiento de 32 GB', 100.00, 25, 'A'),
    ('Disco SSD', 'Unidad de estado solido de 500 GB', 600.00, 15, 'A');

-- Ajustar la secuencia del rol tras insertar IDs manuales
SELECT pg_catalog.setval(
    pg_get_serial_sequence('public.rol', 'id_rol'),
    (SELECT MAX(id_rol) FROM public.rol),
    true
);
```

Script datos_prueba.sql. Script de roles y productos de ejemplo; se aprecia la carga inicial y el ajuste de la secuencia.

```sql
-- ============================================
-- PERMISOS DE POSTGRESQL PARA EL VPS
-- Ejecutar como administrador de PostgreSQL
-- ============================================

GRANT USAGE ON SCHEMA public TO app_inventario;

GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLE public.productos TO app_inventario;

GRANT SELECT
ON TABLE public.rol, public.usuario
TO app_inventario;

GRANT USAGE, SELECT
ON ALL SEQUENCES IN SCHEMA public
TO app_inventario;
```

Script permisos.sql. Sentencias GRANT para la cuenta técnica app_inventario.

```sql
-- ============================================
-- SISTEMA WEB DE INVENTARIO
-- ESTRUCTURA DE BASE DE DATOS - POSTGRESQL
-- ============================================

CREATE TABLE public.rol (
    id_rol INTEGER GENERATED BY DEFAULT AS IDENTITY,
    nombre VARCHAR(50) NOT NULL,
    CONSTRAINT rol_pkey PRIMARY KEY (id_rol),
    CONSTRAINT rol_nombre_key UNIQUE (nombre)
);

CREATE TABLE public.usuario (
    id_usuario INTEGER GENERATED BY DEFAULT AS IDENTITY,
    usuario VARCHAR(50) NOT NULL,
    password VARCHAR(255) NOT NULL,
    estado CHAR(1) NOT NULL DEFAULT 'A',
    id_rol INTEGER NOT NULL,
    CONSTRAINT usuario_pkey PRIMARY KEY (id_usuario),
    CONSTRAINT usuario_usuario_key UNIQUE (usuario),
    CONSTRAINT usuario_estado_check
        CHECK (estado IN ('A', 'I')),
    CONSTRAINT usuario_id_rol_fkey
        FOREIGN KEY (id_rol) REFERENCES public.rol(id_rol)
);

```

Script schema.sql: tablas rol y usuario. Definiciones CREATE TABLE con claves, estados y relación por id_rol.

```sql

    CREATE TABLE public.productos (
        id_producto INTEGER GENERATED BY DEFAULT AS IDENTITY,
        nombre VARCHAR(100) NOT NULL,
        descripcion VARCHAR(255),
        precio NUMERIC(12,2) NOT NULL,
        stock INTEGER NOT NULL DEFAULT 0,
        estado CHAR(1) NOT NULL DEFAULT 'A',
        CONSTRAINT productos_pkey PRIMARY KEY (id_producto),
        CONSTRAINT ck_productos_estado
            CHECK (estado IN ('A', 'I')),
        CONSTRAINT productos_nombre_check
            CHECK (BTRIM(nombre) <> ''),
        CONSTRAINT productos_precio_check
            CHECK (precio >= 0),
        CONSTRAINT productos_stock_check
            CHECK (stock >= 0)
    );
```

Script schema.sql: tabla productos. Campos y restricciones del catálogo, incluyendo precio y stock no negativos.

17. Conclusiones

1. Se logró integrar y publicar el sistema web en una máquina virtual Ubuntu Server en Microsoft Azure, combinando Nginx como proxy inverso, Apache Tomcat 10, Java 17 y PostgreSQL 16, lo que permitió aplicar en un caso real la administración de servicios, el acceso remoto por SSH y el control de permisos.
2. La organización del backend en Servlets, modelos y DAO separa la interfaz, la lógica de negocio y la persistencia, lo que facilita el mantenimiento y la incorporación de nuevas funciones.
3. El modelo relacional protege la consistencia del inventario mediante claves primarias y foráneas, restricciones que impiden precios y existencias negativos, y estados que permiten la eliminación lógica de registros.
4. Las pruebas funcionales confirmaron el inicio y cierre de sesión, la consulta, el registro, la edición y la eliminación de productos, así como la diferenciación de permisos entre los perfiles Administrador y Operador.
5. Las medidas de seguridad aplicadas —credenciales en variables de entorno fuera del repositorio, contraseñas con PBKDF2-SHA256, protección CSRF y un usuario de base de datos con permisos limitados— siguen el principio de mínimo privilegio.
6. Para una operación en producción aún es necesario habilitar HTTPS, activar y revisar el firewall, documentar con capturas las pruebas reportadas y verificar la restauración de un respaldo.
7. La implementación del sistema web permitió comprender la importancia de integrar diferentes tecnologías, como Java, PostgreSQL y Ubuntu Server, para desarrollar una aplicación funcional que facilite la administración de productos y el control de inventarios.
8. El uso de GitHub como herramienta de control de versiones facilitó la organización del trabajo en equipo, permitiendo que cada integrante desarrollara sus actividades de manera independiente y contribuyera a la integración del proyecto.
9. Las pruebas realizadas con los perfiles de Administrador y Operador permitieron comprobar el funcionamiento de las principales operaciones del sistema, así como la importancia de establecer permisos específicos para proteger la información y evitar acciones no autorizadas.
10. El desarrollo de este proyecto fortaleció los conocimientos adquiridos durante el curso de Sistemas Operativos, especialmente en la administración de servidores Linux, configuración de servicios, conexión con bases de datos y despliegue de aplicaciones web.